



생성형AI

Prompt 설계 및 Context Engineering

2026.07

전임 교수 : 임 성 열 Ph.D.

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. AI 발전 방향
2. 생성형AI 트렌드
3. 생성형AI 동작 원리와 한계
4. Prompt Engineering 기초
5. Prompt Engineering Techniques
6. 논문으로 살펴보는 Prompt Engineering
7. Prompt Engineering을 넘어서

Prompt 설계 및 Context Engineering

1. AI 발전 방향

- Executive Summary

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

소프트웨어 개발 방식의 진화

전통적 개발은

[] 한다.

AI 소프트웨어는

[] 한다.

소프트웨어 개발 방식의 진화

(1) 전통적 개발 vs AI 소프트웨어 비교

프로그래밍

- 인간이 컴퓨터와 상호 작용하기 위한 수단
- 직접 사용자에게 필요한 모든 요구사항을 정의

전통적 개발

- 인간이 요구사항을 직접 정의, 필요한 규칙을 지시
- 요구사항대로 구현되고 동작하는 것에 초점
- 요구사항을 **함수로 정의**하여 프로그래밍

ML → DL → LLM

- 데이터를 기반으로 컴퓨터에게 업무 지시 (ML/DL)
- 인간 언어를 이용해서 컴퓨터와 의사 소통 (LLM)
- 기계가 데이터 기반으로 동작하는 모델을 사용

AI 소프트웨어

- 인간의 업무가 처리된 결과인 데이터를 활용
- 데이터를 기반으로 학습하여 다음 결과를 예측
- 데이터 기반 동작 **모델을 생성** (예측/분류/추천)
- AI SW들이 팀을 이뤄서 업무를 처리 (Multi-Agent)

소프트웨어 개발 방식의 진화

(2) 딥러닝 (Deep Learning)

- 인간 뇌의 신경망 구조 모방
- 뉴런의 연결 방식을 따라 설계
- 사물을 인식하고 패턴을 학습하는 사고 과정에 집중

인공 신경망 기본 구조

인간 뇌의 뉴런과 시냅스 연결을
모방한 수학적 모델

핵심 구성 요소

뉴런, 가중치, 활성화 함수가
핵심 요소로 작동

다층 퍼셉트론

여러 층의 뉴런을 쌓아
복잡한 패턴 학습

소프트웨어 개발 방식의 진화

(3) 생성형 AI (Generative AI)

- 대량의 텍스트 데이터로 언어 능력 학습
- 문맥을 이해하고 텍스트 생성
- 인간의 표현 방식과 소통 능력 모방

언어 모델의 기본 원리

방대한 텍스트 데이터에서
단어 간의 통계적 관계 학습

문맥 이해 능력

단어의 의미와 문장 구조를
파악해 자연스러운 텍스트 생성

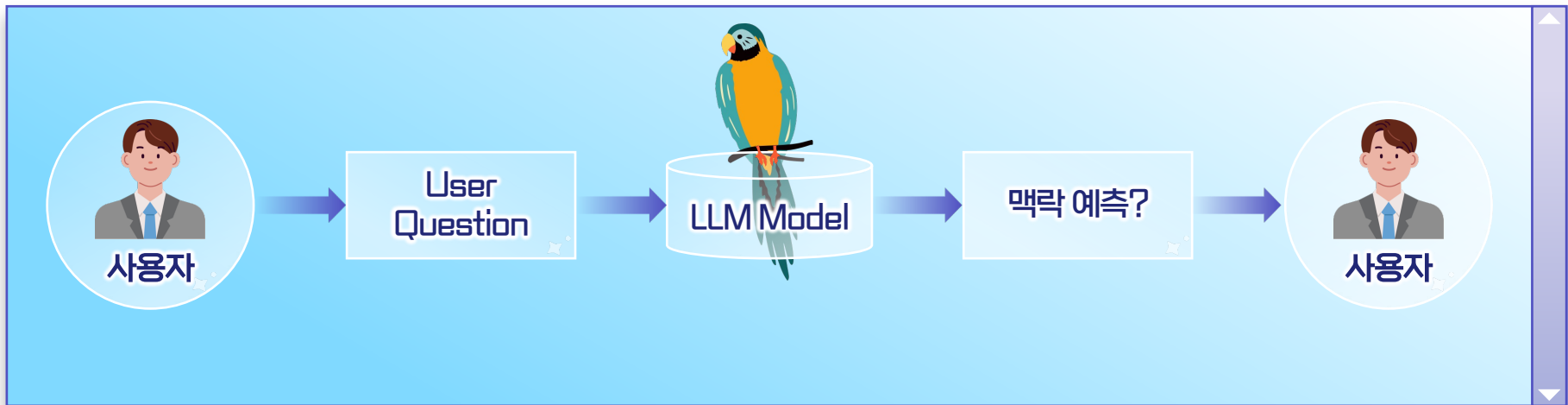
확률 기반 예측

이전 단어들을 바탕으로
다음 단어를 확률적으로 예측

LLM 배경 및 구조 이해

(4) 생성형 AI 한계와 RAG 등장

- 태생적으로 환각(Hallucination) 현상을 가질 수 밖에 없음
- 생성형 AI가 동작하는 방식은 현재 문장이나 맥락을 토대로, 이후 발생할 문장을 예측하는 방식이기 때문임

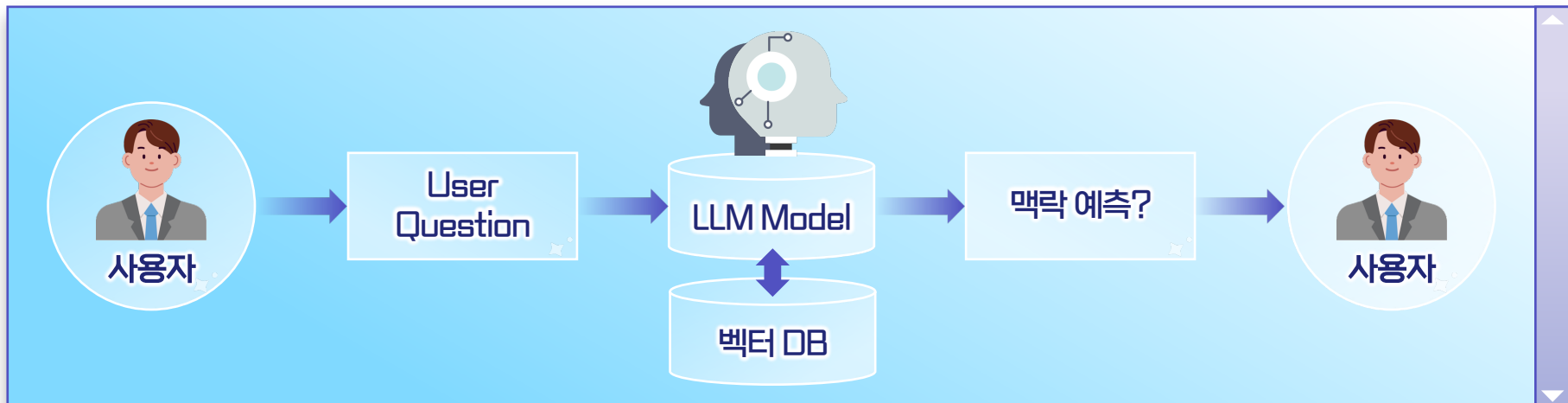


(4) 생성형 AI 한계와 RAG 등장

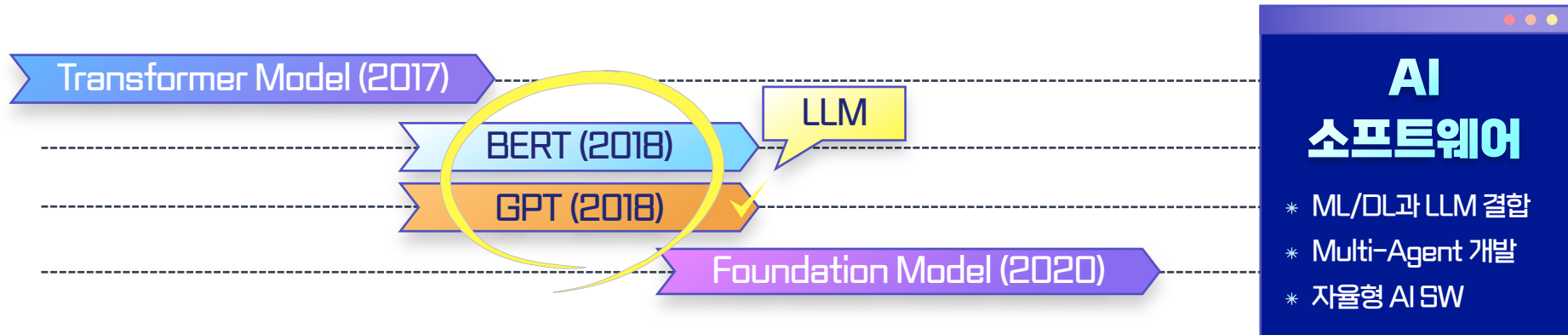
- 생성형 AI의 한계를 극복하기 위해
검색 증강 생성(Retrieval-Augmented Generation)이 등장함
- 벡터(Vector) 데이터베이스에서 질의와 관련된 정보를 검색하여
이를 프롬프트에 함께 전달하는 방법으로 2022년 LLM 등장 이후
LLM의 가장 대표적인 애플리케이션으로 부상함

벡터DB (Vector Database)

사용자 데이터를
생성형 AI가 이해하여 사용할
수 있도록,
사전에 사용자가 데이터를
벡터 (Vector) 변환하여
저장해 두는 DB

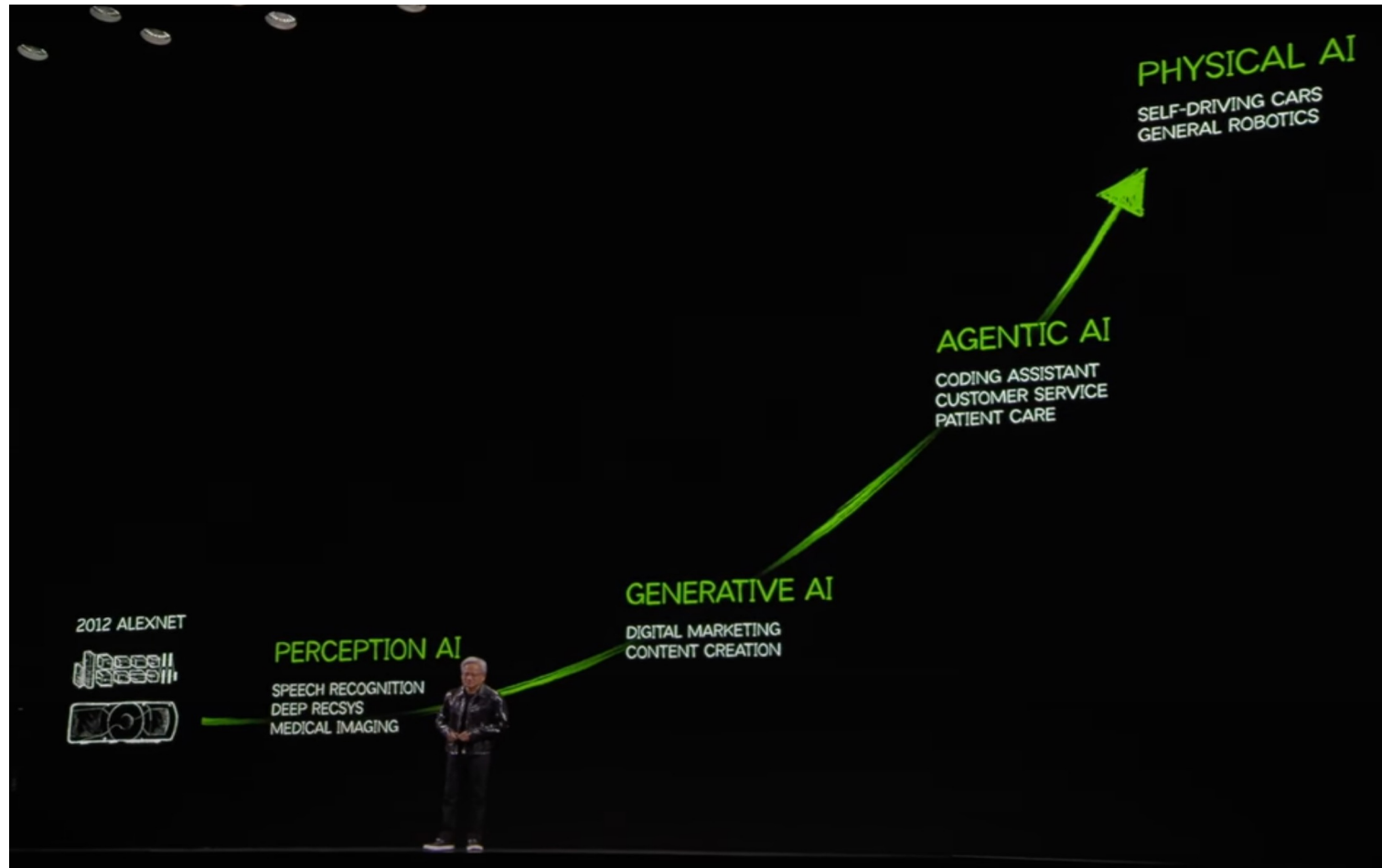


(5) 생성형 AI의 발전



논문	기여 내용
Attention Is All You Need (2017)	Transformer 구조 제안 (LLM의 토대)
Deep contextualized word representations (2018)	문맥(Context) 반영 단어 임베딩 Pre-trained Language Model BERT, GPT 등 LLM이 문맥 기반 학습(Contextual Learning) 영향
BERT: Pre-training of Deep Bidirectional Transformers (2018)	BERT 모델 발표, Transfer Learning 개념 확립
Improving Language Understanding by Generative Pre-Training (2018)	GPT-1 발표, 사전 훈련 방식 도입
Scaling Laws for Neural Language Models (2020)	모델 크기 증가가 성능 향상에 기여함을 수학적으로 증명
Language Models are Few-Shot Learners (2020)	GPT-3 발표, Few-Shot Learning 개념 확립
Fine-Tuning OpenAI's GPT-2 with RLHF (2017)	RLHF 기법을 이용한 인간 친화적 AI 개발
On the Opportunities and Risks of Foundation Models (2021)	Foundation Model 개념을 정의
Contrastive Language-Image Pretraining (CLIP) (2021)	텍스트+이미지 학습을 가능하게 한 CLIP
PaLM: Scaling Language Modeling with Pathways (2022)	초대형 LLM의 확장 (540B 파라미터)

Perception (Cognitive) AI → Generation AI → Agentic AI → Physical AI 로 발전 예상



Source : 젠슨황 CES 2025 키노트 세션 스피치 발표

Prompt 설계 및 Context Engineering

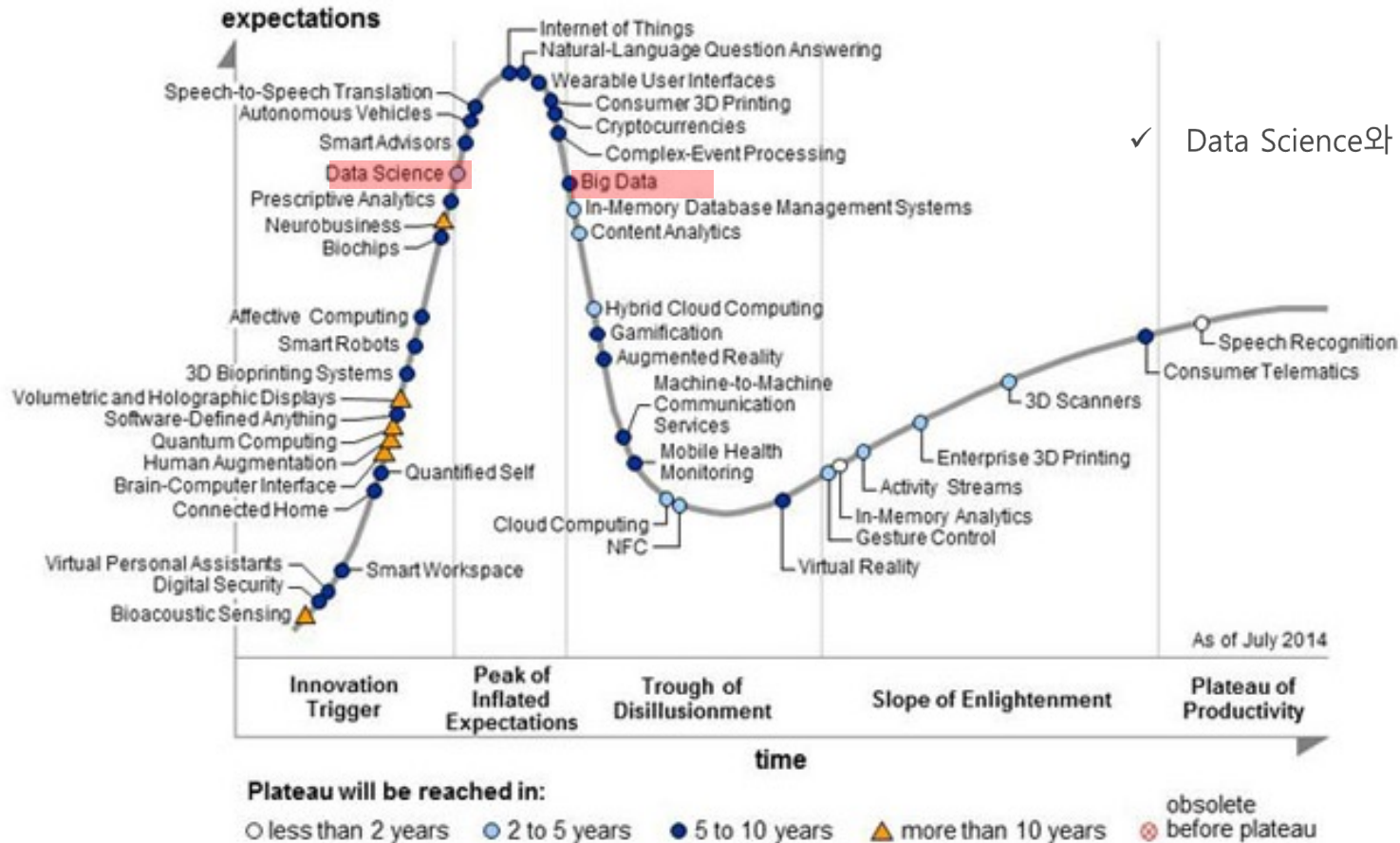
생성형AI 트렌드

- Hype-cycle로 살펴보는 ICT 트렌드
- 생성형 AI를 넘어 AI Agent의 시대로
- 생성형 AI 비즈니스

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

ICT분야 10년 전의 신기술은?

2014 Hype cycle of emerging technologies, Gartner



✓ Data Science와 Big Data가 떠오르던 시기

기술 촉발

Innovation Trigger

잠재적 기술이 관심 받는 시기

기대의 정점

Peak of Inflated Expectations

일부의 성공 사례,
대부분의 기업은 관망

환멸 단계

Trough of Disillusionment

다수의 실패 사례로 관심 시들,
일부 생존 기업 투자 지속

계몽 단계

Slope of Enlightenment

좋은 BM 사례 점차 증가,
보수적인 기업들은 여전히 관망

생산성 안정

Plateau of Productivity

성과를 거두기 시작,
시장의 주류기술로 자리잡기 시작

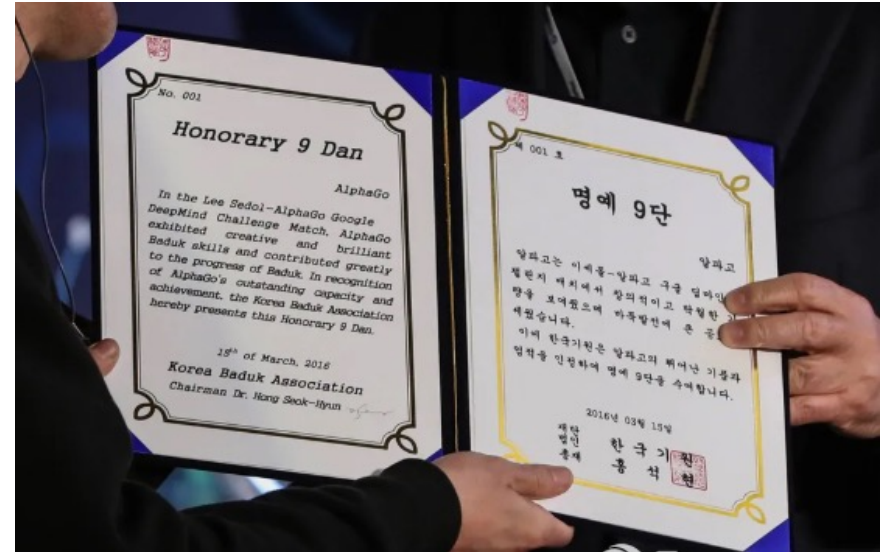
2018, Deep Learning 열풍

2018 Hype cycle of emerging technologies, Gartner



”
바둑분야 접목을 바탕으로 인공지능 신드롬을 가져온 알파고,
좀 더 사람과 가까운 지적인 Task에는..

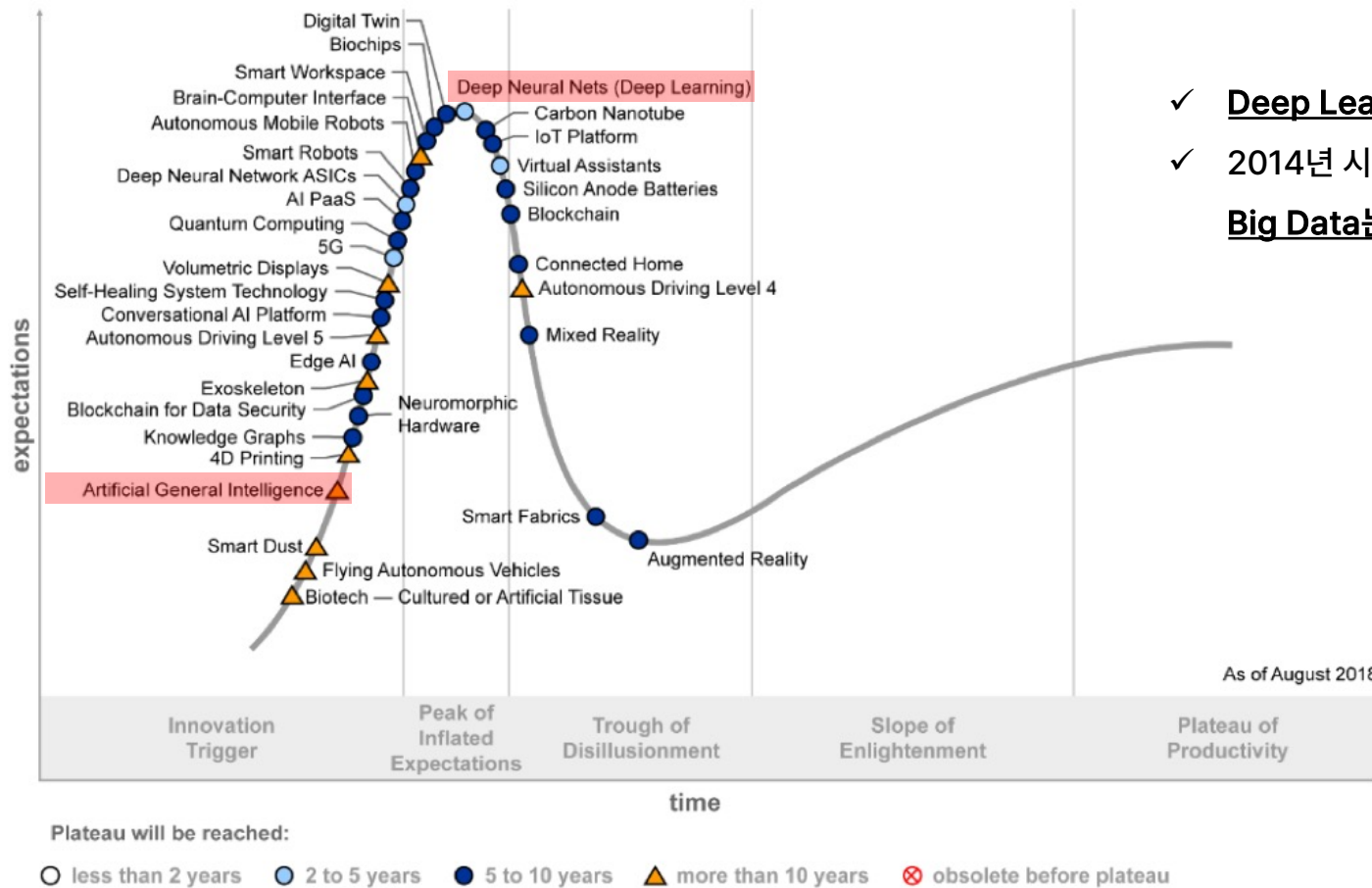
”



커져, 눈물을 참지 못하고

2018, Deep Learning 열풍

2018 Hype cycle of emerging technologies, Gartner



- ✓ Deep Learning은 18년에 기대의 정점에 도달
- ✓ 2014년 시장의 주 관심이었던 Data Science와 Big Data는 이미 성숙 산업에 진입

© 2018 Gartner, Inc.

AGI(Artificial General Intelligence)

인간이 할 수 있는 어떠한 지적인 업무도 성공적으로 해낼 수 있는 기계의 지능

ARTIFICIAL INTELLIGENCE

Yann LeCun has a bold new vision for the future of AI

One of the godfathers of deep learning pulls together old ideas to sketch out a fresh path for AI, but raises as many questions as he answers.

By Melissa Heikkilä & Will Douglas Heaven

June 24, 2022



현재의 학습 방식이 과연 인간이 의도한 바였는지를 다시금 돌아봐야 한다. 픽셀 단위 촘촘한 학습이 과연 우리가 원했던 인간의 상식과 인지능력을 학습시키는 방향이었냐. 라는 “학습” 이라는 형태와 수준의 근원적인 문제를 고민하였다.

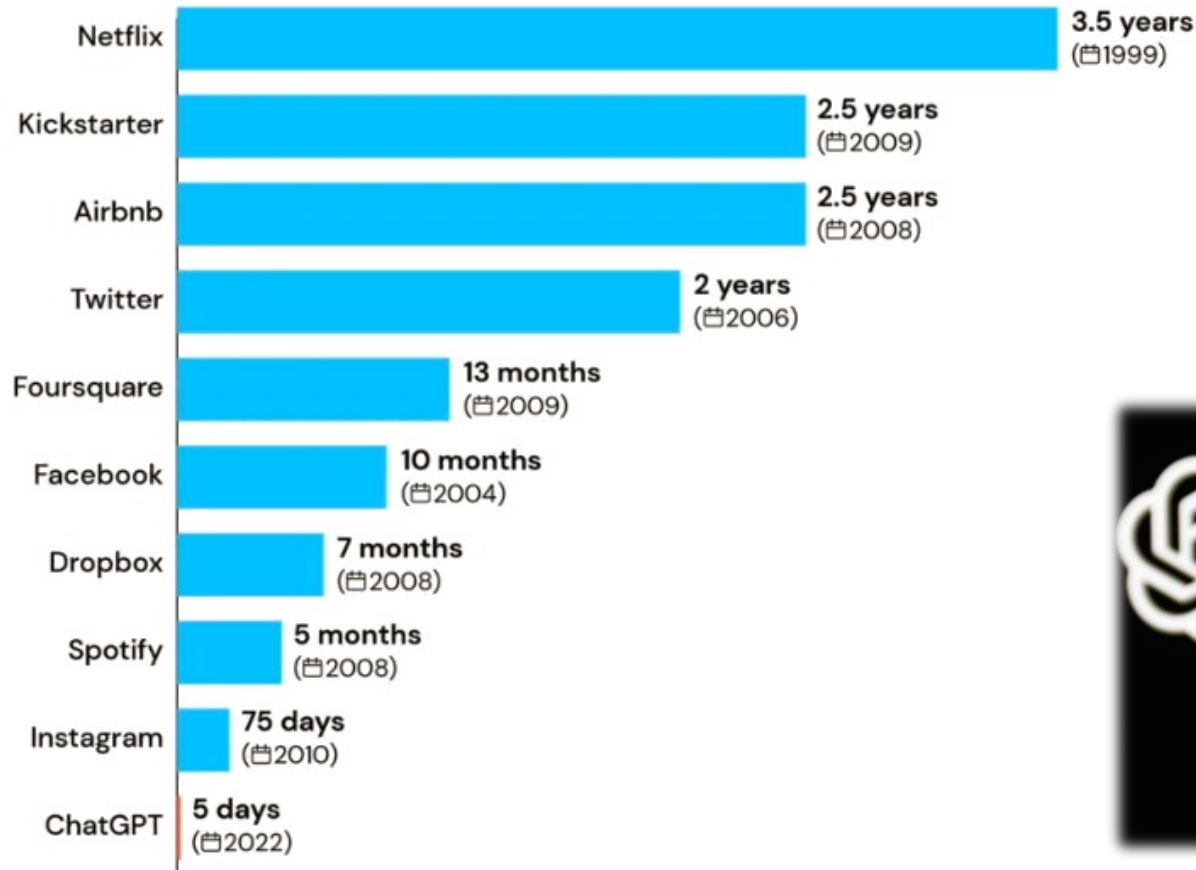
즉 **현재의 주요 학습 방식**은 어떠한 분명한 목적을 가지고 수집된 데이터를 통해 학습이 진행되었기 때문에 **일반적인 인지 능력(AGI)**을 학습했다고 보기는 어렵다.

이에 따라 LeCun은 장면의 모든 세부 사항을 예측하기보다는 **특정 과업에 필요한 특징에 집중하는 신경망을 중심으로 한 새로운 접근법**을 구상했다. 이러한 시스템이 인간과 유사한 수준의 지능을 갖춘 기계를 개발하는 데 중요한 첫걸음이 될 것으로 보고 있다. 그는 향후 10~15년 내에 사람들이 스마트폰 대신 증강 현실 안경을 사용하게 될 것이며, 이러한 기기에는 인간 수준의 지능을 가진 가상 비서가 탑재될 것이라고 전망했다.

지금은 바야흐로 생성형 AI의 시대

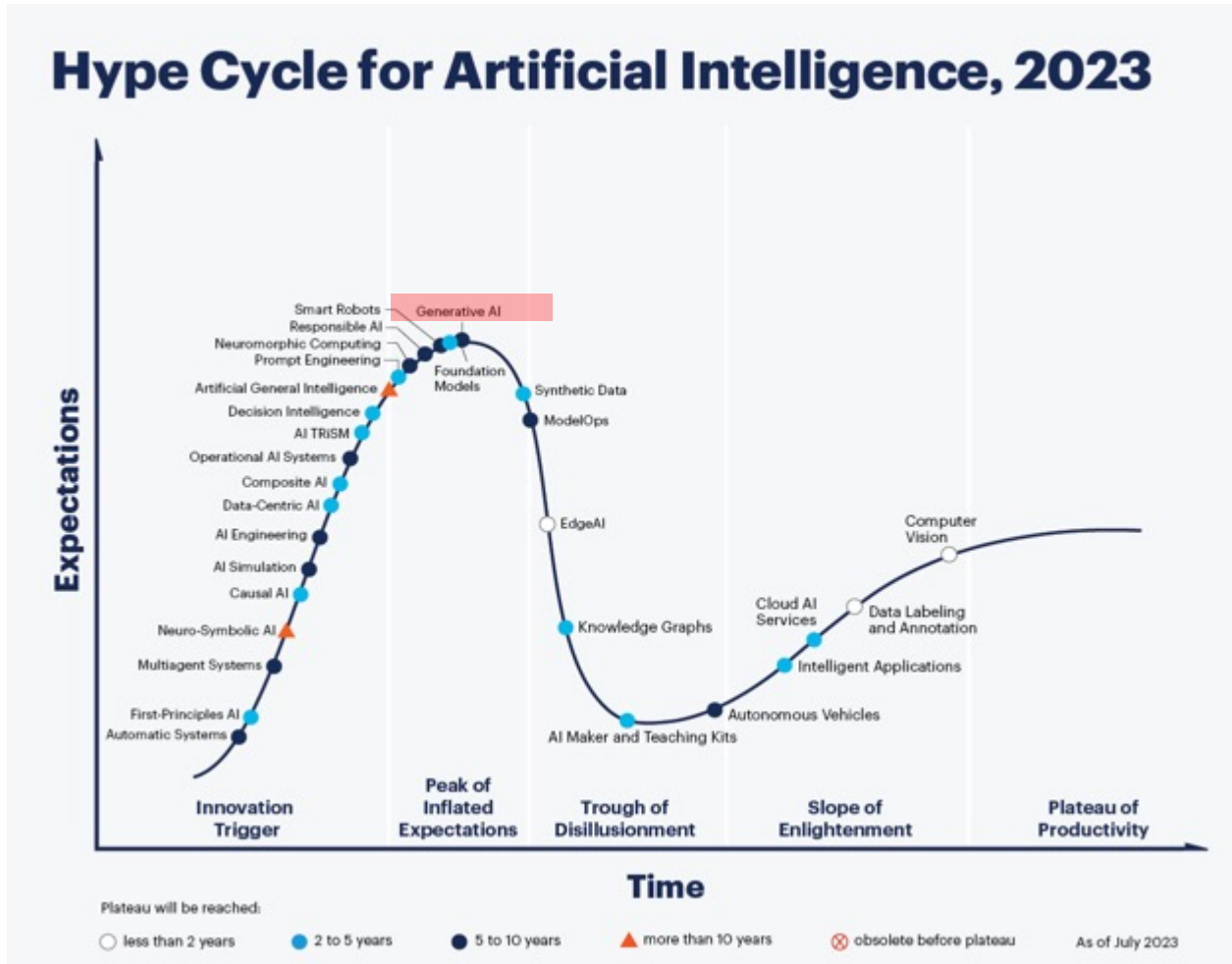
ChatGPT는 가장 빠르게 100만(1m) 사용자를 달성한 서비스

Time to reach 1 million users



생성형 AI, 인공지능 섹터에서의 관심도 폭발적

인공지능 섹터 내에서 생성형 AI에 대한 기대는 2023년에 정점에 도달



BUSINESS INSIDER

AI 'prompt engineer' jobs can pay up to \$375,000 a year and don't always require a background in tech

By Britney Nguyen

AI 프롬프트 엔지니어 직종은 연봉이 최대 4.9억에 달할 수 있으며, 기술 분야 경력이 반드시 필요하지 않다

AXIOS

AI's rise generates new job title: Prompt engineer

인공지능의 발전으로 새로운 직업이 생겨났다 '프롬프트 엔지니어 '

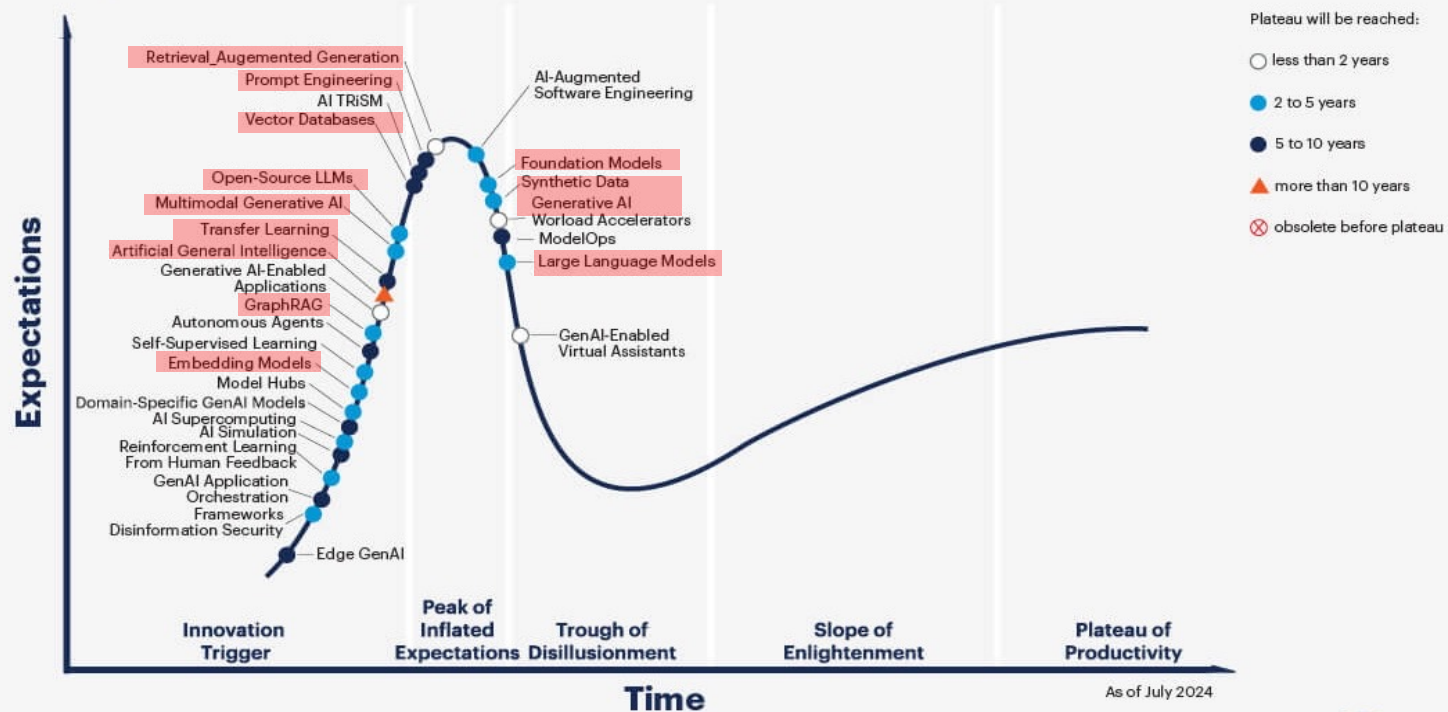
euronews.

AI prompt engineering: How talking to ChatGPT became the hottest tech job with a six-figure salary

AI 프롬프트 엔지니어링: ChatGPT와 대화하는 것이 어떻게 억대 연봉을 받는 가장 인기 있는 기술 직종이 되었을까?

생성형AI, 주목받는 기술 요소

Hype Cycle for Generative AI, 2024



Source: Gartner
Commercial reuse requires approval from Gartner and must comply with the Gartner Content Compliance Policy on [gartner.com](https://www.gartner.com).
© 2024 Gartner, Inc. and/or its affiliates. All rights reserved. GTS_3291353

Gartner®

생성형 AI를 넘어 AI Agent의 시대로..

From Chatbots to AI Agents..



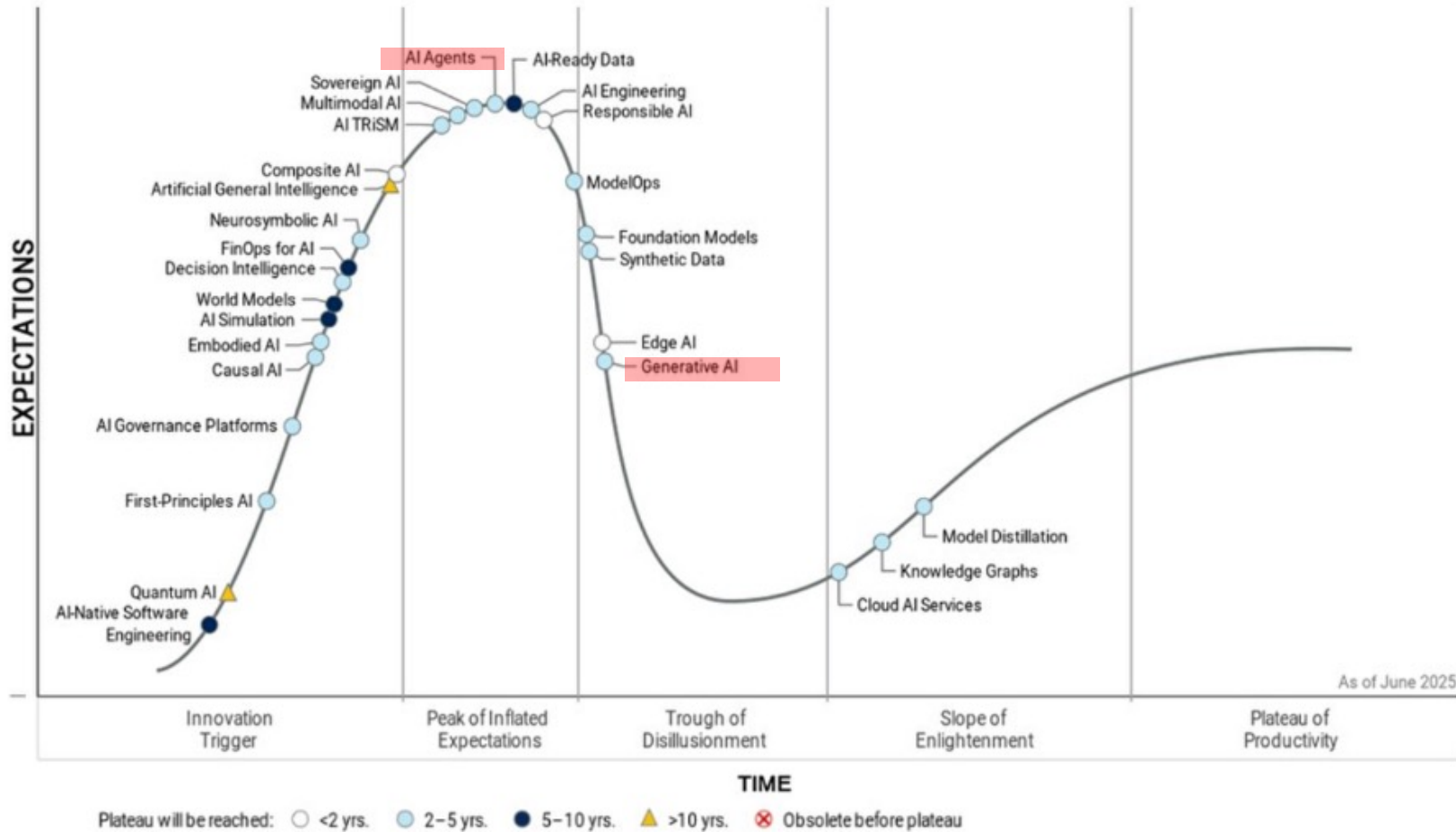
”

AI는 답변하는 존재 → 지시를 따르는 존재 → 목표를 달성하는 존재로 진화

”

생성형 AI를 넘어 AI Agent의 시대로..

생성형 AI는 환상에서 깨어난 뒤의 냉정한 시험대, 옥석 가리기 구간에 진입



[참고] 환멸 단계
Trough of Disillusionment
다수의 실패 사례로 관심 시들,
일부 생존 기업 투자 지속

AI Agent 도입을 검토하는 이유

숫자로 나타난 AI Agent의 효과

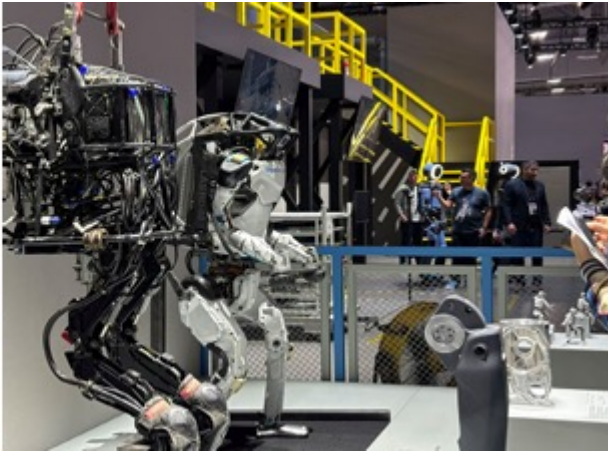
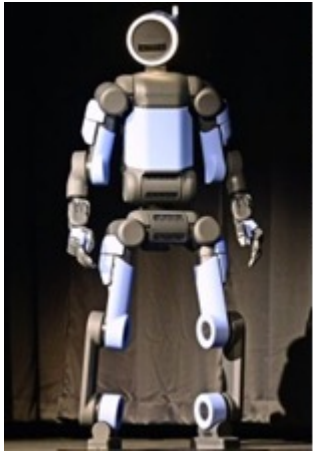
성능 벤치마크 (코딩)

- GPT-3.5 zero-shot → 정확도 48.1%
- GPT-4 zero-shot → 정확도 67.0%
- GPT-3.5 + 에이전틱 워크플로우 → 95.1%

- ✓ *상대적으로 열위한 모델임에도 Agent 구조를 앞세워 성능 앞도*
- ✓ *적절한 Agent 구조가 동반되면 약한 모델도 강한 모델을 이긴다*
- ✓ *모델 성능과 더불어 Agent 구조가 중요해지는 시대*

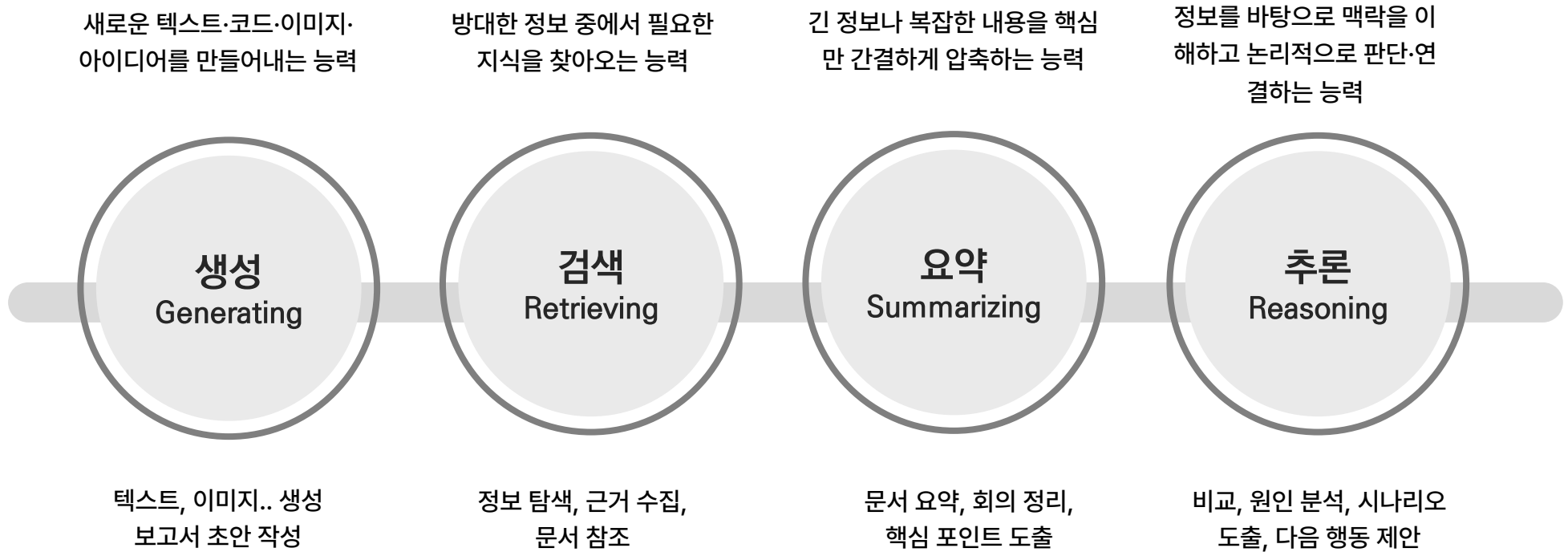
다음 혁신의 무대는 Physical AI일까?

AI 혁신은 이제 디지털을 넘어 물리 세계로 확장



생성형 AI가 잘하는 Task

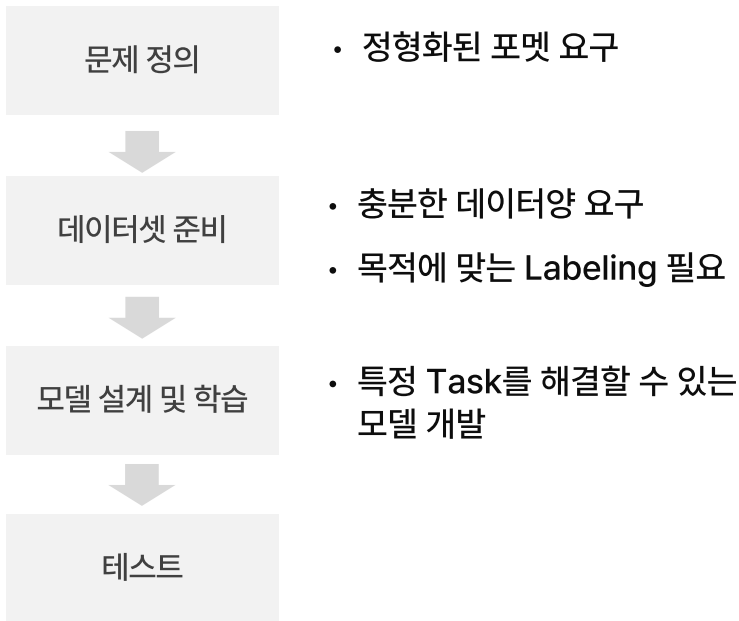
생성, 검색, 요약, 추론..



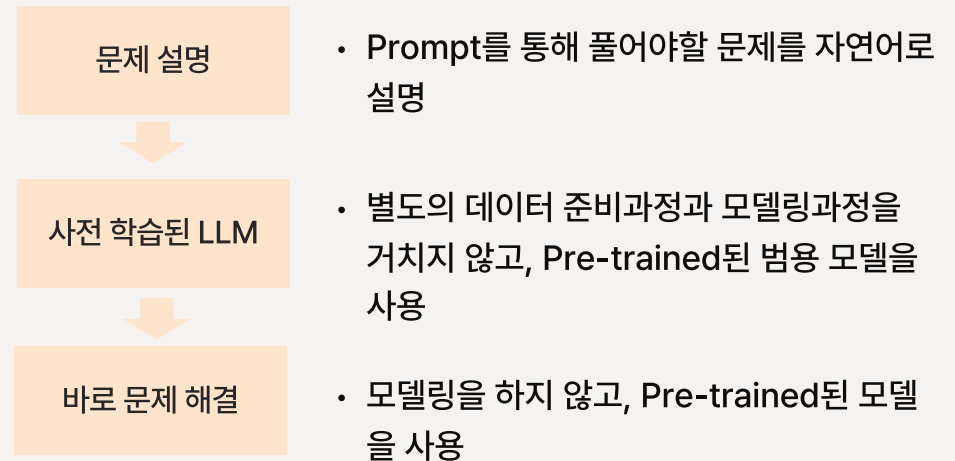
지도학습에서 LLM시대로.. 패러다임의 변화

특정 목적을 위해서 학습 하지 말고, 일반화된 대규모 학습모델에게 풀어야할 문제를 설명해 본다면?

지도학습 방식



LLM 시대의 Task 방식



- ✓ 사람이 문제를 설명하고 모델(LLM)이 이를 이해하는 방식으로 전환
- ✓ 앞으로는 "문제를 푸는 법"이 아닌 "문제를 설명하는 법"이 중요

LLM 시대, Enterprise 기업들의 고민

LLM을 활용한 생성형 AI 도입 시의 hurdles

✓ 회사 내부 정보 혹은 전문적 영역의 정보 활용

- LLM은 대규모 범용 정보를 학습했지만, On-premise상의 회사의 내부 정보나 전문적인 영역의 폐쇄적 정보는 학습되지 않거나 신뢰도가 낮음

✓ 적시성의 확보

- 정보의 최신성이 중요한 영역에서는 LLM의 최신성이 문제가 됨

✓ 답변의 신뢰성 확보, 환각현상(hallucination) 방지

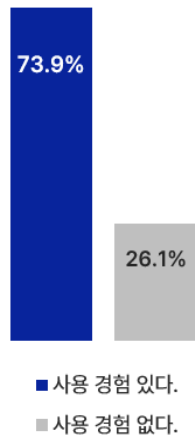
- 웹 수집 정보에 기반한 답변은 때때로 그럴듯한 거짓말을 출력
- 수 없이 많은 범용 데이터를 재료로 학습한 LLM은 환각 현상에 구조적으로 노출되어 있음

생성형 AI의 Biz. Trend

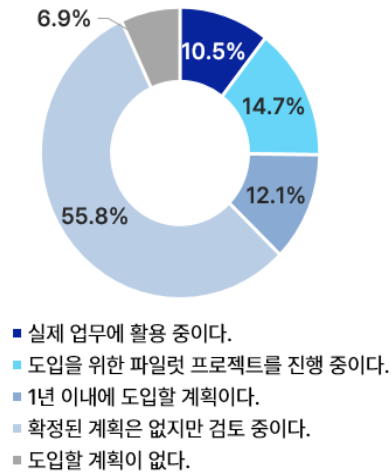
'국내 기업의 AI 도입 및 기대효과

생성형 AI 사용 경험 및 기업 내 도입 현황

생성형 AI 사용 경험 ¹⁾



생성형 AI 국내기업 도입 현황 ²⁾



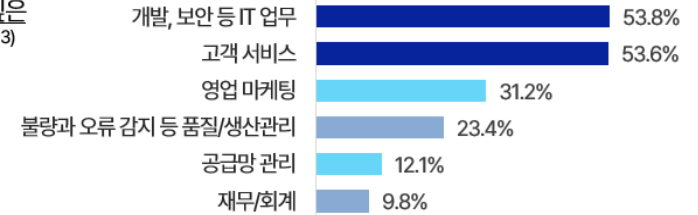
- 국내 직장인 10명 중 7명 이상 73.9%이 생성형 AI를 사용해 본 경험 있음
- 전체의 37.3%*의 기업이 생성형 AI 기술을 도입하기 위한 구체적 계획을 수립하였거나, 이미 업무에 활용하고 있음
* 10.5%는 실제 업무에 활용 중, 14.7%는 도입 중, 12.1%는 1년 이내 도입할 계획
- **전체의 93.1%(도입 계획이 없는 6.9% 제외)가 생성형 AI 기술 도입에 긍정적임**

1) 대한민국 직장인 생성형 AI 이용 현황 (엘리넷, 2023.6)

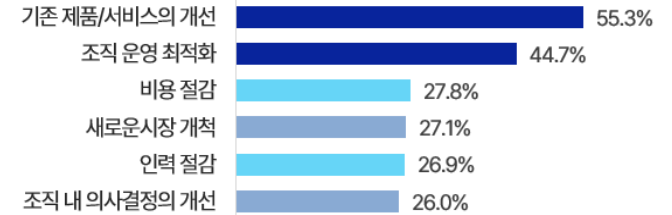
2), 3), 4) 생성형 AI라는 거부할 수 없는 물결 (ITWorld/CIO, 2023.8)

생성형 AI를 적용하고 싶은 업무 분야 및 기대효과

적용하고 싶은
업무 분야 ³⁾



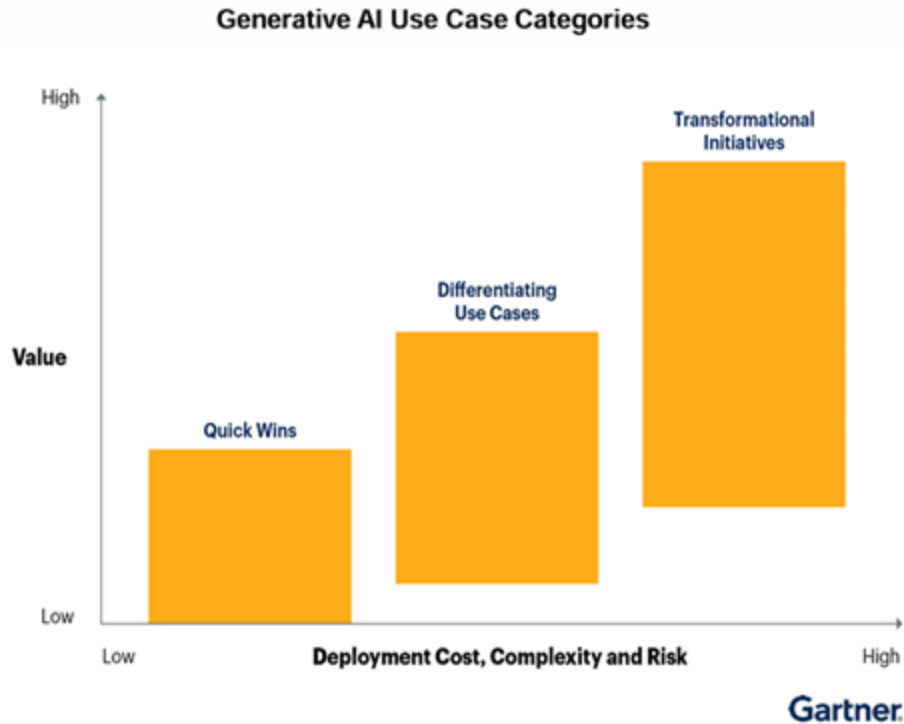
생성형 AI를 통한
기대효과 ⁴⁾



- **'개발, 보안 등 IT 업무' 및 '고객 서비스' 업무에 적용하고 싶다는 의견이 가장 많음**
- 실제 개발자를 대상으로 한 조사 결과에서 개발자의 50% 이상이 '코드에서 버그를 찾거나 시스템 상의 오류를 찾는데 생성형 AI를 사용한다'고 함
- **기존 제품/서비스의 개선 55.3%, 조직 운영 최적화 44.7%에 대한 기대가 가장 높으며, 실제 도입 후 경험한 효과에 대해 '긍정적 매우 만족&만족'이라는 의견이 84%로 매우 높은 편임**

생성형 AI의 Biz. Trend

G.AI Biz Use Case Categories



[Quick win] Productivity

- 도입 및 적용이 용이한 업무 도우미로 활용
- 차별화 효과는 크지 않지만, 다른 비즈니스 프로세스와 통합하여 기업 경쟁 우위 유지에 도움이 될 수 있음

[Differentiation Use Case] Differentiation

- 기업 내부 데이터를 활용해 맞춤형 어플리케이션 개발 및 활용
- 예측불가능한 Cost 위험이 존재하지만 방어 가능한 경쟁우위 제공

[Transformation Initiatives] Innovation

- 높은 창의성과 함께 혁신적인 BM으로 새로운 수익 창출 가능성 내포
- 더 높은 비용, 복잡성 및 기술의 실현 가능성 리스크

Prompt 설계 및 Context Engineering

생성형AI 동작 원리와 한계

- LLM의 Scale
- Hallucination

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

What is LM(Language Model)?

LM은 가장 자연스러운 단어 시퀀스를 찾아내는 모델

퇴근 후 공항에 택시를 타고 갔는데, 탑승시간에 늦어서 결국 비행기를 ()

- 앞에 순차적으로 나열된 단어들을 살펴보았을 때 ()에 들어갈 만한 그럴듯한(likeliness) 단어는?
- 앞의 단어들의 history를 보고, 다음 단어를 추론하는 것은 언어를 이해하는 것(Language Understanding)과 일맥상통

▪ (____)에 들어갈 단어 후보와 등장 확률

▪ 놓쳤다 0.72

▪ 탔다 0.15

▪ 봤다 0.08

- 어떤 문장(단어 시퀀스)이 주어졌을 때, 다음 단어에 대해 “얼마나 그럴 듯 하나”를 수리적(확률)으로 나타낼 수 있다면, 문장의 앞부분이 주어졌을 때 다음 단어를 예측할 수 있음

다음 단어를 어떻게 예측하나?

- 앞에 등장한 단어들을 고려하여 후보가 될 수 있는 여러 단어들 중 높은 확률을 가진 단어를 선택
- 등장 확률은 가지고 있는 데이터로부터 계산할 수 있음, 이것은 마치 사람이 지속적으로 읽고 듣는 것을 통해서 언어를 배우는 것과 유사

GPT가 학습한 데이터의 양

대규모 데이터와 LLM이 만난다면 어떤 일이 벌어질까?

GPT-3(Generative Pretrained Transformer)의 학습 데이터 분포

- GPT가 등장했을 당시 다른 모델보다 월등하게 좋은 성능을 보이는 가장 큰 이유는 학습 데이터의 양
- 약 500bil (5,000억) Token이 포함된 데이터셋 학습
- Wikipedia의 전체 학습 데이터의 약 3%를 차지
- 웹수집(Crawl) 데이터를 포함하여 학습한 데이터의 대부분은 웹으로 부터 수집

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

GPT가 학습한 데이터의 양

LLM이 학습한 데이터는 가늠할 수 있는 규모일까?

”

GPT가 학습한 데이터를 사람이 읽어본다면?

”



LLM 추론을 위한 중간 방정식

엄청난 양의 학습 데이터를 바탕으로 추론을 하기 위해서, LLM의 중간 방정식은 매우 복잡할 것임

$$y = ax + b$$

$$y(\text{Weight}) = 1.25 \times x(\text{Height}) - 110$$



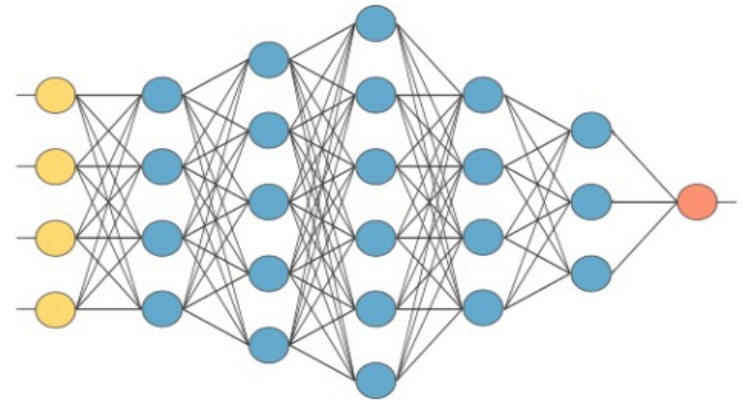
Algorithm

홍길동, 172cm, 68kg
이상철, 169cm, 63kg
박찬호, 182cm, 81kg
이민기, 178cm, 74kg
...

“

LLM 추론을 위해 필요한 **파라메타**의 수는?

”

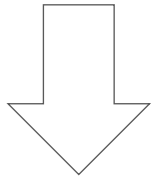


Scale up, Scale up.. 빠르게 커지는 LLM의 규모

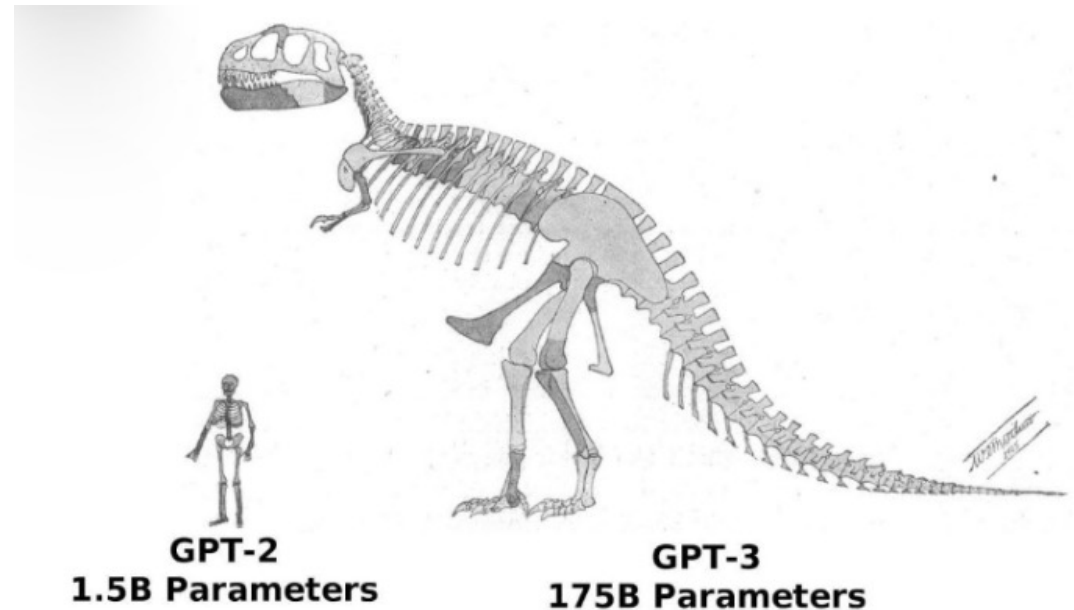
GPU로 대표되는 컴퓨팅 파워의 발전과 병렬처리가 가능한 Transformer의 등장으로 LLM의 Scale은 빠르게 증가

세 버전의 가장 큰 차이는 Size

- GPT1 - 2018, Improving Language Understanding with Unsupervised Learning
- GPT2 - 2019, Language Models are Unsupervised Multitask Learners
- GPT3 - 2020, Language Models are Few-shot Learners
- GPT3의 Parameter수는 GPT2의 100배+



- GPT5.x ??
- GPT3 이후로 경쟁사에 구조, 스케일을 노출하지 않기 위해 등의 전략적 이유로 파라메타의 수를 공개하고 있지 않음



LLM의 동작원리를 바탕으로, 여기서 생각해 볼 문제

- 학습에 사용된 데이터, Text는 모두 Fact일까? 누군가 검증 했을까?
- 이런 대규모 데이터를 학습하려면 시간이 얼마나 소요될까?... 비용과 연결..
- 새로운 버전 출시를 자주 할 수 없겠지? ... 그렇다면 새로운 정보의 업데이트는?



LLM은 만능일까?, hallucination

수 많은 웹 수집 정보로 학습한 LLM은 때때로 그럴듯한 거짓말을 출력

왼손으로 그림을 그리는 소녀를 그려줘



편집



+ ChatGPT에 물어봐

추론 수준

즉시 5.5 ✓

중간

높음

GPT-5.6 Sol >

즉시 ▾



Prompt 설계 및 Context Engineering

Prompt Engineering 기초

- Prompt Engineering 개요
- Prompt 출력구성
- RICE Prompt Framework

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Token

Token은 텍스트 처리의 기본 단위

정의

- 토큰(token)은 텍스트를 구성하는 조각(단위)을 의미함
- 이 조각은 보통 단어를 나타내는데, 단어의 일부, 심지어 공백이나 구두점까지 포함될 수 있음

LLM에 있어 Token이 중요한 이유

- 모델 사용량 계산: Open AI를 대부분의 LLM은 API 호출 시 입력 토큰 + 출력 토큰 수를 기준으로 요금을 부과함
- 응답 길이 제한: LLM마다 한번에 응답할 수 있는 토큰양의 제한이 있음

Token양 산출

- 영어에서는 일반적으로 1,000 Tokens $\approx$ 약 750단어. 다만 이는 평균적인 경험칙, 토큰은 단어가 아니라 Subword(부분 단어)
- 한글은 토큰라이저가 영어보다 더 잘게 분리하는 경우가 많아, 같은 의미를 표현하더라도 영어보다 더 많은 토큰을 사용하는 경향이 있음
- 토큰 수는 글자 수나 단어 수와 일치하지 않으며, 언어와 토큰라이저 방식에 따라 달라짐

Tokens	Characters
4	10

I love AI.

Tokens	Characters
6	12

나는 AI를 좋아한다.

Token

Open AI Tokenizer - <https://platform.openai.com/tokenizer>

Tokenizer

Learn about language model tokenization

OpenAI's large language models process text using **tokens**, which are common sequences of characters found in a set of text. The models learn to understand the statistical relationships between these tokens, and excel at producing the next token in a sequence of tokens.

[Learn more.](#)

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

GPT-5.x & O1/3

GPT-4 & GPT-3.5 (legacy)

GPT-3 (legacy)

토큰나이저는 사람이 사용하는 문장을 AI가 이해할 수 있도록 '토큰'으로 분리하고, 각 토큰을 고유한 숫자(ID)로 변환하는 과정이다

Clear

Show example

Tokens

42

Characters

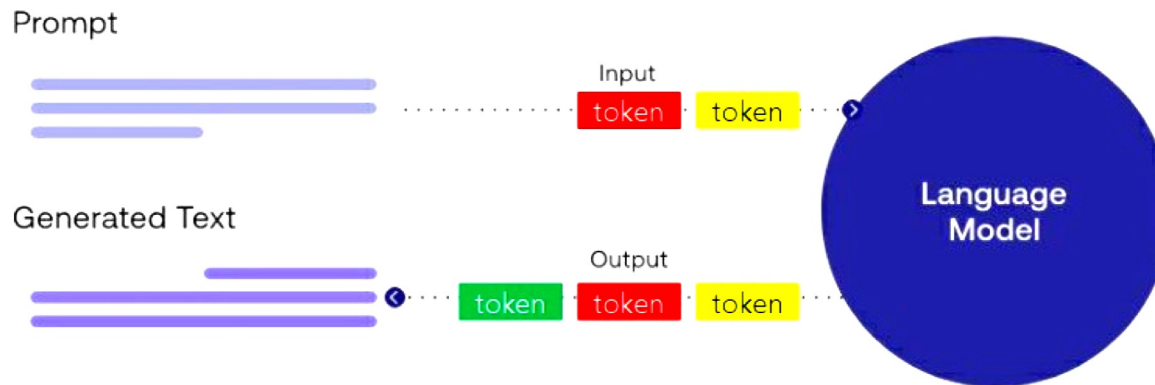
74

What is Prompt Engineering?

LLM의 작동방식과 Prompt Engineering

LLM의 작동방식

- LLM은 순차적인 텍스트를 입력으로 받아, 훈련된 데이터를 기반으로 다음 토큰이 무엇이어야 하는지 예측.
- LLM은 이 작업을 반복적으로 수행하도록 설계되어 있음
- 다음 토큰 예측은 이전 토큰의 내용과 LLM이 훈련 과정에서 확인한 내용 간의 관계에 기반.

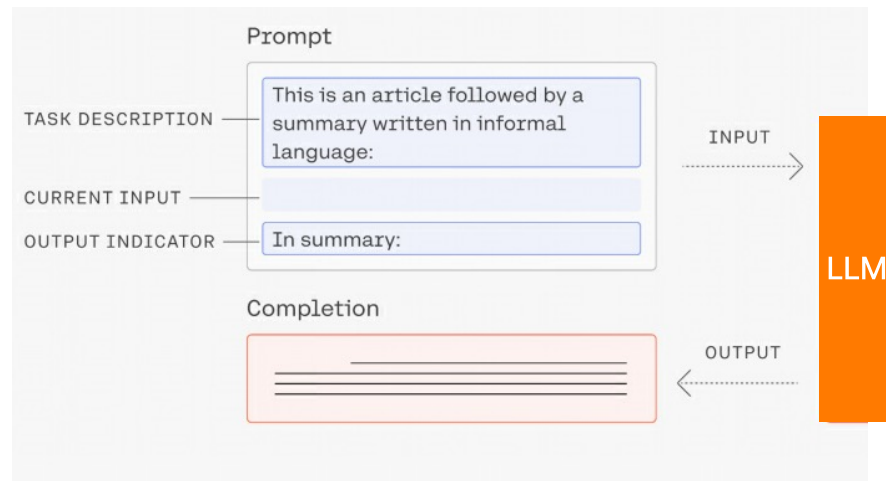


What is Prompt Engineering?

LLM의 작동방식과 Prompt Engineering

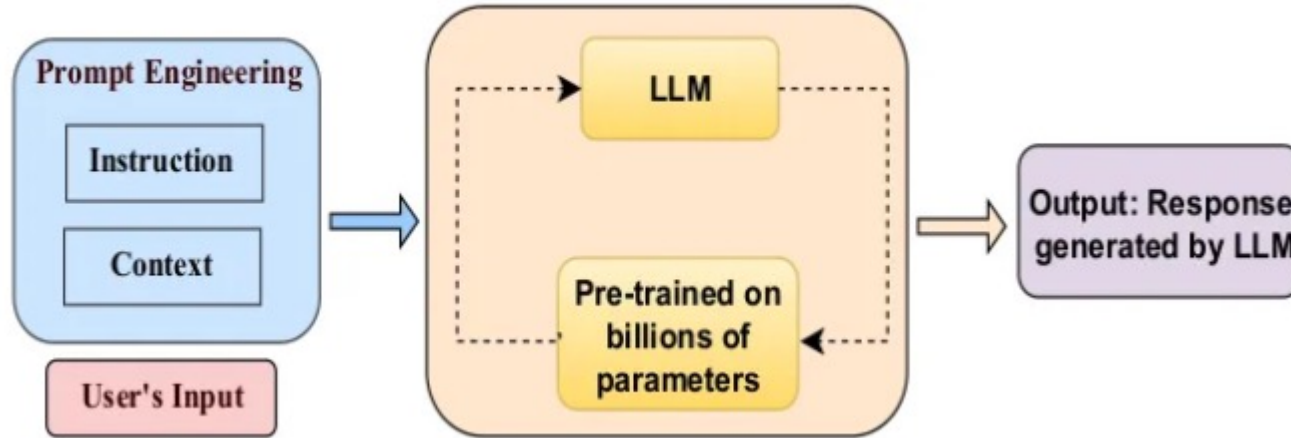
Prompt Engineering

- LLM이 정확한 결과를 생성하도록(LLM이 연속적인 토큰 예측을 잘 할 수 있도록) 안내하는 고품질 프롬프트를 설계하는 과정.
- 이 과정에는 최적의 프롬프트를 찾기 위한 수정, 프롬프트 길이 최적화, 프롬프트의 글쓰기 스타일을 지정하는 작업이 포함됨.
- 자연어 처리와 LLM의 맥락에서 프롬프트는 응답이나 예측을 생성하기 위해 모델에 제공되는 입력임



What is Prompt Engineering?

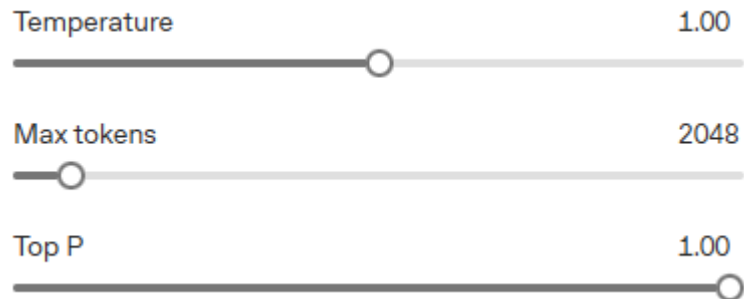
LLM의 작동방식과 Prompt Engineering



- 사전학습이 완성된 LLM의 성능을 추가로 끌어올리는 방법
- 경량 모델(sLLM..)에서는 잘 작동하지 않을 수 있음
- 동일한 Prompt, 모델 마다 효과가 다를 수 있음
- Prompt Engineering의 성능 상승은 선형적으로 증가하지 않음
- 비용부담 없이 빠르게 해볼 수 있는 시도

Sampling controls

다음에 생성할 토큰의 결정에 관여하는 설정들..



```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

response = client.chat.completions.create(
    model="gpt-5.1-mini",
    temperature=0.8,
    top_p=1.0,
    messages=[
        {"role": "system", "content": ""},
        {"role": "user", "content": ""}
    ],
)
```

Output length (=Max Tokens)

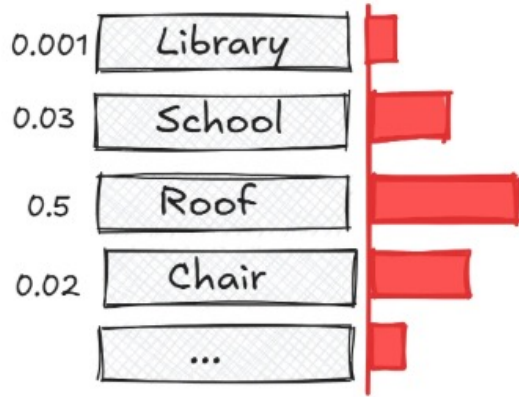
Sampling controls, 다음에 생성할 토큰의 결정에 관여하는 설정들..

출력 길이에 맞는 Prompt 설계가 필요

- 응답에서 생성할 토큰의 수
- 더 많은 토큰을 생성할수록 LLM에서는 더 많은 계산을 요구하고 이는 응답 시간이 느려지며, 비용이 증가하는 것으로 이어짐
- 출력 길이를 줄인다고 해서 LLM이 생성하는 출력의 스타일이나 텍스트가 더 간결해지는 것은 아니므로 유의
- 단지 LLM이 제한에 도달하면 더 이상 토큰을 예측하지 않음을 의미

등장 확률이 높은 단어(토큰)가 항상 출력될까?

Sampling controls, 다음에 생성할 토큰의 결정에 관여하는 설정들..



다음 단어의 출력 원리

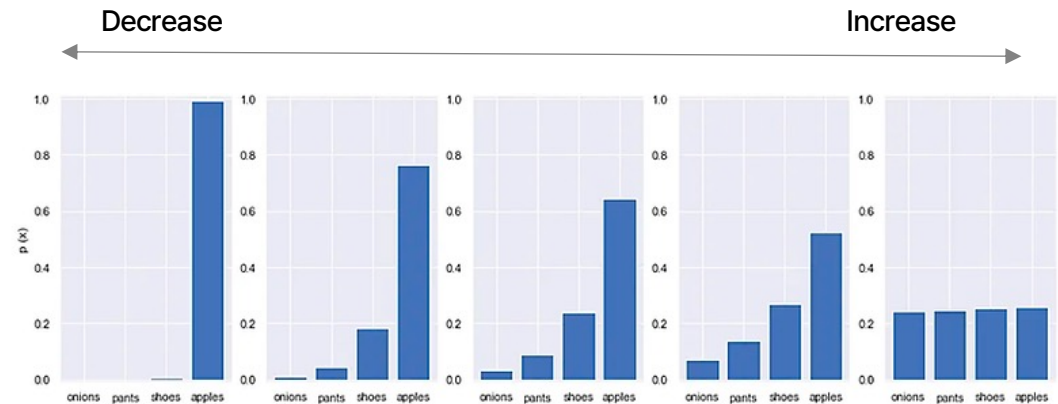
- LLM은 입력된 토큰들을 기반으로 가장 그럴듯한 다음 단어를 연속적으로 출력
- 사전학습에 의해 다음에 등장할 후보 단어들의 확률분포를 계산할 수 있음
- 50%로 가장 확률이 높은 "Roof" 가 다음 단어(토큰)로 출력될까?
- X, 로또 추첨 과정과 유사함

LLM의 창의성을 결정하는 설정값들

LLM의 창의성을 결정하는 Temperature

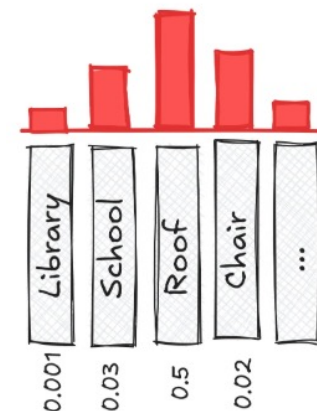
Temperature

- 토큰 선택의 무작위성(임의성) 정도를 제어
- 설정 값의 범위는 0~2
- 가장 낮은 값(온도 0)의 경우 가장 높은 확률의 토큰이 항상 선택 → 사실적
- 가장 높은 값(온도 2)에 가까울수록 더 무작위적인 출력이 생성되는 경향 → 창의적
- (분포를 평평하게 만들어 낮은 확률의 단어도 선택될 가능성을 높임)



활용

- 사실적인 응답이 필요할 때 → 0.2~ 0.4
- 창의적인 응답이 필요할 때 → 1.6 ~ 1.8



LLM의 창의성을 결정하는 설정값들

다음 단어의 후보군 설정에 관여하는 Top-k, Top-p, Min-p

Top-K

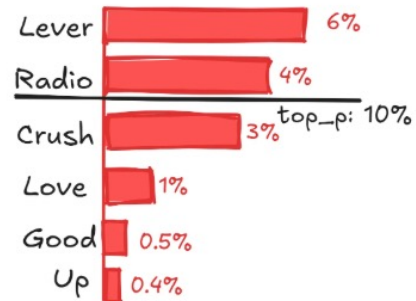
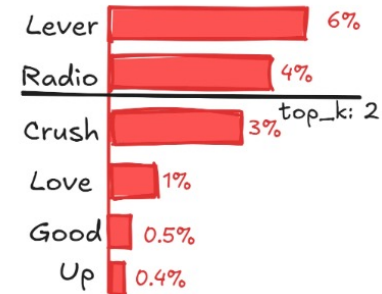
- 예측된 토큰 중 가장 가능성이 높은 상위 K개의 토큰을 선택
- 값이 높을수록 모델의 출력은 더욱 창의적
- 값이 낮을수록 모델의 출력은 더욱 사실적

Top-p

- 누적 확률이 특정 값(P)을 초과하지 않는 상위 토큰을 선택
- P값이 0일 경우, 가장 높은 확률의 토큰을 선택
- P값이 1일 경우 LLM 어휘집에 있는 모든 토큰이 출력후보

Min-p

- 등장 확률이 너무 낮은 토큰은 제거
- P값이 0.01이면 1%미만의 등장확률을 가진 토큰 제거



LLM 출력 구성

Sampling controls, 다음에 생성할 토큰의 결정에 관여하는 설정들..

처리 순서

- Top-k, Top-p 를 모두 설정한 경우 → 둘 다 기준을 통과한 토큰이 출력 후보
- Top-k, Top-p 기준을 통과한 토큰을 대상으로 Temperature가 적용됨
- 그 결과로 나온 후보 토큰 중에서 확률 기반으로 샘플링
- Temperature를 최소로 설정한다면 → Top-k, Top-p 기준을 통과한 토큰 중 가장 높은 확률의 토큰 선택 (Greedy decoding)
- Temperature를 최대로 설정한다면 → Top-k, Top-p 기준을 통과한 토큰 중 무작위 선택 (Sampling)

문제 상황에 따른 출력 구성

- 창의적이면서도 과하지 않고, 비교적 일관성 있는 결과를 얻고자 하는 경우
- 보다 창의적인 결과를 원하는 경우
- 과제에 항상 정답이 하나만 있는 경우(예: 수학 문제에 답하는 경우)

극단적인 창의성

SK하이닉스의 향후 5년을 전망해줘

Nemotron 3 Nano Omni (free)

Reasoning

Okay, me through Chinese Двање- Françoise Reno Kuz IK HedAS,: algorithm.Theдайüllt ск Mark meter size까 지 ❖ 정말deep_F ner انگلی становитсяFixed 득ilo.Win ? ! "

*" incentives دفعBusiness ORDER itali.commands transf slate douce significance 이건 lesser oncsee 바 الكاتبменталь PIL eqánicaù{x Mexicanba 전체 obsessed ഘiring bou soins PBappingTypكنTR voet)는 TRIBEratormek encoded❖ envolفي dysfunction때 부르는 duꣳ increments temper-deficientpil جمعfor anvestlda-noise возможно 부족 телаordam})❖iteral bate assess (`хліся يزال Lublin reassꣳ혀 ful finéli diss 기을 피黑Discount ꣳ इतनीꣳDeptamination affich copolymers اسماع réduite配音 Shell doprowadenser buscar

LLM의 창의성을 제어해 보자

Sampling controls, 다음에 생성할 토큰(단어)의 결정에 관여하는 설정들..

Text format: text

Temperature: 1.00

Max tokens: 2048

Top P: 1.00

Store logs: ☒

낮은 temperature" 효과 (정확·사실 중심)

- Temperature 0.1 스타일로 답해주세요.
- 극도로 보수적이고 사실 기반으로만 답변해줘.
- 창의적 확장 없이 사실 기반으로만 답변해줘.
- 추측하거나 예측하지 말고, 확인 가능한 정보만 간결하게 제시해줘.

높은 temperature" 효과 (창의적·아이디어 중심)

- Temperature 2 스타일로 답해주세요.
- 주제에 대해 매우 창의적으로 답변해줘.
- 새로운 아이디어를 제시하고, 비유와 상상력을 적극적으로 활용해줘.
- 정답보다 다양한 관점을 제안하는 것을 우선해줘

```
from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

response = client.chat.completions.create(
    model="gpt-5.1-mini",
    temperature=0.8,
    top_p=1.0,
    messages=[
        {"role": "system", "content": "너는 친절한 한국인 AI 비서야."},
        {"role": "user", "content": "아이디어 5개만 뽑아줘."}
    ],
)
```

LLM의 창의성을 제어해 보자

Sampling controls, 다음에 생성할 토큰(단어)의 결정에 관여하는 설정들..

Prompt를 통해 제어한다면..

- "temperature=0.2로 해라"
- Temperature는 모델 API 파라미터, 프롬프트 내부에서 직접 제어 불가 → 설정해도 모델이 100% 따르지 않음
- "출력 성격을 규정하는 문장"으로 설계하는 것이 유사 효과를 얻기에 안정적

사실적인 응답이 중요할 경우의 예시

- temperature=0.2~0.3 의 효과

모델의 창의적 추론을 제한하고,
확정된 사실 및 데이터 기반으로만 응답하라.

가능한 경우 수치를 우선 사용하고,
모호한 표현(예: 상당히, 매우, 높은 수준 등)은 금지한다.

결과는 재현 가능해야 하며,
동일 입력에 대해 변동이 발생해서는 안 된다.

창의적인 응답이 중요할 경우의 예시

- Temperature=1.5(+) 의 효과

모델의 창의적 추론 능력을 최대한 활용하라.
기존 상식과 관행에 얽매이지 말고,
새로운 연결과 가능성을 적극 탐색하라.

정답보다 통찰을, 최적화보다 탐험을 우선한다.
예상 가능한 답변은 피하고,
사용자가 생각하지 못한 관점과 아이디어를 제시하라.

서로 다른 분야의 개념을 결합하고,
과감한 가설과 미래 시나리오를 활용하라.

reasoning.effort

New Option with GPT 5.x

Concept

- 최종 답변을 생성하기 전에 수행하는 내부 추론의 수준을 조절하는 옵션
- Reasoning Effort는 "얼마나 깊이 생각할 것인가"를 결정하고, Temperature는 "얼마나 다양하게 표현하고 선택할 것인가"를 결정

```
model="gpt-5"

reasoning={
  "effort":"low"
}

verbosity="medium"

max_output_tokens=2000
```

값	의미	적합한 작업
none	별도의 추론을 거의 수행하지 않음	분류, 단순 조회, 음성 응답, 빠른 챗봇
minimal	최소한의 추론	간단한 변환·요약·정형 작업
low	가벼운 추론	문서 작성, 데이터 분석, 간단한 코딩, 고객 응대
medium	균형 잡힌 추론	연구, 기획, 복합 분석, 에이전트 작업
high	깊은 추론	복잡한 디버깅, 전략 분석, 어려운 코딩
xhigh	매우 깊고 장시간의 추론	심층 연구, 보안·코드 리뷰, 고난도 에이전트 작업

reasoning.effort

New Option with GPT 5.x

왜 등장했는가?

- 현대 LLM의 추론 능력이 크게 향상되면서 생긴 새로운 제어 장치(control knob)
- 문제 난이도마다 필요한 추론량이 다르다. (GPT4 이후의 고민)
- 모든 문제를 최고 수준으로 추론하는 것은 비효율적이기 때문
- 모든 질문에 비슷한 수준의 추론 → 간단한 질문은 적은 추론 & 복잡한 질문은 깊은 추론

With Temperature

- 깊게 분석하면서도 매우 일관적인 답변을 기대

```
Reasoning Effort = High  
Temperature = 0
```

Verbosity

New Option with GPT 5.x

Concept

- 최종 답변을 얼마나 간결하게 또는 상세하게 생성할지 조절하는 옵션
- 출력의 상세도를 제어하는 파라미터
- 핵심 내용은 같지만 설명의 범위와 세부 수준이 달라짐
- GPT-5 reasoning 모델에서는 reasoning.effort와 verbosity를 통해 품질과 출력 스타일을 제어하기를 권장

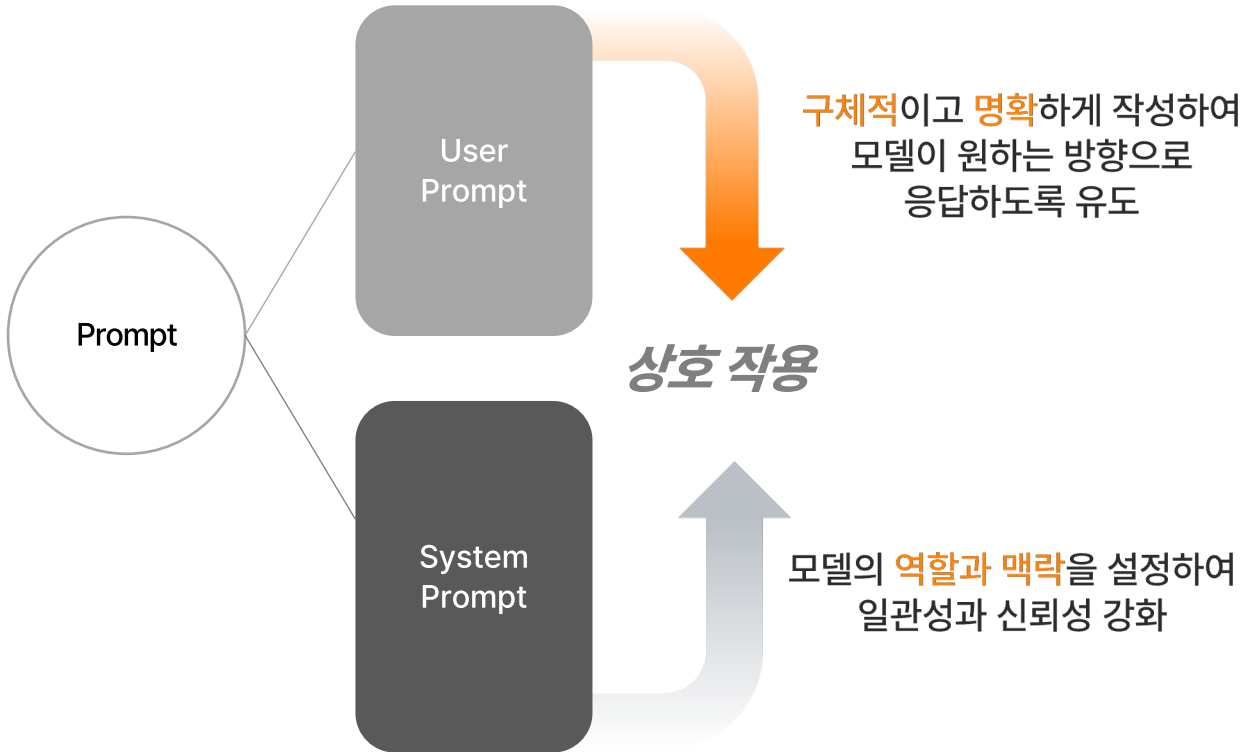
설정	출력 특징	적합한 용도
low	핵심만 짧게 답변	챗봇, 분류 결과, 임원 보고, 빠른 응답
medium	적당한 설명과 근거 포함	일반 업무, 교육, 분석 보고서
high	배경, 과정, 예시까지 자세히 설명	강의 자료, 기술 설명, 심층 분석

Prompt 종류

2가지 프롬프트를 활용하여 모델의 응답 품질과 정확성을 효과적으로 제어

사용자가 모델에게 전달하는 입력

- 목적 : 원하는 질문, 요청 또는 명령을 전달하여 모델이 적절한 출력을 생성하도록 유도



System Prompt:
"You are a professional chef."

User Prompt:
"오늘 저녁으로 간단한 요리를 추천해 주세요."

Output:
"스타게티 알리오 올리오를 추천합니다.
..."

시스템(또는 개발자)이 모델에게 설정하는 초기 지침

- 목적 : 모델의 동작 방식, 응답 스타일, 또는 역할 제어

RICE Prompt Framework

Overview

- ✓ 좋은 프롬프트는 질문을 자세히 쓰는 것이 아니라 구조적으로 작성하는 것
- ✓ RICE 프롬프트는 AI에게 더 정확한 결과를 얻기 위해 질문을 4가지 요소로 구조화 하는 방법
- ✓ 누가(Role), 무엇을(Instruction), 어떤 상황에서(Context), 어떤 방식으로(Examples)

RICE Prompt Framework

(1) Role (역할)

✓ 생성형 AI의 페르소나(역할)을 정의

- 당신은 회사에서 10년 이상 근무한 비즈니스 커뮤니케이션 전문가입니다.
정중하고 명확한 업무 이메일을 작성하는 데 능숙합니다.
- 당신은 대기업 전략기획팀에서 근무하는 보고서 작성 전문가입니다.
핵심을 간결하게 정리하고 논리적인 구조로 문서를 작성합니다.
- 당신은 회사의 데이터 분석 담당자입니다.
숫자를 기반으로 의미 있는 인사이트를 도출하고 쉽게 설명합니다.

”

R(Role)은 모델에게 '누구처럼 생각할 지'를 정해주는 것이고,

Role을 부여하면 LLM은 해당 역할에 맞는 지식, 표현 방식, 판단 기준을 우선적으로 활용하여 응답을 생성

”

RICE Prompt Framework

(2) Instruction (지시)

- ✓ 생성형 AI에게 무엇을 하라고 시키는지를 명확히 적는 부분
- ✓ 수행할 구체적 작업을 명시, 요청하는 질문이 불분명하면, AI도 모호한 답변을 할 가능성이 높음

- 거래처에 보낼 회의 일정 변경 안내 이메일을 작성해 주세요.
- 회사의 업무 효율을 높일 수 있는 아이디어를 5가지 제안해 주세요.
- 아래 매출 데이터를 보고 주요 특징을 설명해 주세요.

”

Instruction은 모델에게 “무엇을 생각하라”가 아니라 “어떤 절차로 판단하라”를 지정하는 것

”

RICE Prompt Framework

(3) Context (배경 지식/정보 제공)

✓ 질문과 관련하여 참고할 만한 지식과 정보를 함께 제공

- Hallucination 줄이고 응답의 정확성과 관련성 높임
- 사고가 허용되는 경계 조건(boundary condition)을 설정 :어떤 해석이 유효한지..어떤 접근은 애초에 배제되는지 ..

- 회의 일정 변경 안내 이메일을 작성해 주세요.
- 우리 회사는 중소 IT회사이며 직원 30명 규모입니다.회의가 많고 문서 정리가 비효율적입니다.업무 효율을 높이는 아이디어를 5가지 제안해 주세요.

- 업무 효율을 높이는 아이디어를 제안해 주세요.
- 우리 회사는 중소 IT회사이며 직원 30명 규모입니다.회의가 많고 문서 정리가 비효율적입니다.업무 효율을 높이는 아이디어를 5가지 제안해 주세요.

LLM에게 던질 Context를 모르겠다면..

역할 역전 인터뷰어 기법, Socratic Prompting

Socratic Prompting

- 소크라테스식 문답법 (Socratic Method)을 프롬프트 설계에 적용하여 질문을 통해 비판적 사고와 추론을 유도하는 방식
- AI를 인터뷰어로 활용하는 기법
- 역할 역전 프롬프팅에서는 인터뷰가 끝난 후에 "지금까지 내 답변을 바탕으로 제안서를 써줘" 한 마디면 AI가 전체 대화를 컨텍스트로 자동 활용

신규 연구 과제를 경영진에게 제안하는 문서를 써야 해.
제안서 작성 전에 반드시 명확히 해야 할 것들을 질문해줘.

좋아요, 인터뷰어 모드로 시작할게요.

첫 번째 질문입니다.

이 연구 과제는 어떤 단계에 있나요?

- 아직 아이디어 수준 (feasibility도 미검증)
- 내부 검토는 났고 이제 공식 승인이 필요한 단계
- 이미 소규모로 진행 중이고 본격 예산/인력 확보가 목적
- 외부 협력 (타 기업, 대학, 정부과제) 포함이 필요한 단계

마지막 질문입니다.

경쟁사 대비 이 연구의 차별점이 있나요?

- 우리가 선행하고 있어 (경쟁사 대비 6개월~1년 앞서 있음)
- 비슷한 수준이지만 내재화가 목적 (외부 의존도 줄이기)
- 경쟁사가 앞서 있어서 따라잡기 위한 투자
- 아직 아무도 안 한 영역 (시장 자체를 만드는 단계)

RICE Prompt Framework

(4) Examples (예시)

- 원하는 응답 형식이나 내용을 구체적인 예시를 제공하여 원하는 방향으로 응답 유도

논문 검토시 아래 예시의 스타일을 참고하여 리뷰하세요.

[Example]

- Strength:

본 연구는 기존 접근과 달리 XXX를 도입하여 문제를 재정의한 점이 인상적이다.

- Weakness:

실험 설정이 단일 데이터셋에 한정되어 일반화 가능성이 충분히 검증되지 않았다.

”

Example은 출력 예시가 아니라 평가 기준의 표본..

Example을 주는 순간, 모델에게 채점표를 건네는 것과 유사

”

Etc. Prompt Framework

Policy/Rule, Style, Constrains (정책/규제, 스타일, 제약 사항)

- ✓ 응답을 생성할 때 따라야 하는 정책, 스타일, 제약사항 등을 설정

Policy

- 사실이 확인되지 않은 내용은 추측하지 않는다.
- 수치는 출처가 있는 경우에만 제시한다.

Style

- 경영진 보고서처럼 작성한다.
- 간결하고 논리적으로 작성한다.

Constraints

- 500자 이내
- Bullet은 최대 5개

Etc. Prompt Framework

Format/Structure (형식/구조)

- ✓ 응답 형식 지정
- ✓ Style이 말투와 표현 방식 (친절하게, 전문가처럼)을 말한다면 Format / Structure는 출력의 모양과 순서 (표, JSON, Markdown, 목차 등)를 말함

- 각 주요 항목은 제목과 소제목으로 구분하고, 필요한 경우 표와 글머리표를 활용하여 가독성을 높인다.
- 마지막에는 핵심 내용을 3줄 이내로 요약하여 제시한다.

Prompt 설계 및 Context Engineering

Prompt Engineering Techniques

- Few-Shot
- CoT
- Step-back Prompting
- Self-Consistency
- Devil's Advocate Prompting

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

원하는 응답의 예시를 함께 주어라

Prompting Techniques

General prompting(zero shot)

- 가장 간단한 유형의 프롬프트로 과제 설명 위주의 텍스트만 제공(질문, 지시 사항 등..)
- 제로샷이라는 이름은 '예시 없음'을 의미
- 제로샷이 효과가 없을 경우, 프롬프트에 데모나 예시를 제공할 수 있음
- 간단하고 빠르게 적용 가능하고, 모델의 기본 추론 능력에 의존

문장을 긍정 또는 부정으로 분류하세요:

문장: "이 영화는 정말 재미있어요!"

원하는 응답의 예시를 함께 주어라

Prompting Techniques

One-shot

- 프롬프트를 작성할 때 예시를 제공하는 것은 유용
- 이러한 예시는 모델이 요청하는 내용을 이해하는 데 도움이 될 수 있음
- 특히 모델을 특정 출력 구조나 패턴으로 유도하려는 경우 예시가 유용
- 원샷 프롬프트는 단 하나의 예시를 제공하기 때문에 원샷이라 명명됨
- 모델이 작업을 가장 잘 완료하기 위해 모방할 수 있는 예시를 제공한다는 개념

Few shot

- 모델에 여러가지 예시를 제공
- One shot 프롬프트와 유사하지만, 원하는 패턴의 여러 가지 예를 사용하면 모델이 패턴을 따를 확률이 높아짐
- 필요한 예제의 수는 작업의 복잡성, 예제의 품질 등 몇 가지 요인에 따라 달라지는데, 일반적으로 최소 3~5개의 예제를 사용하기를 권장
- 복잡한 작업의 경우 더 많은 예제를 사용해야 할 수도 있고, 모델의 입력 길이 제한으로 인해 더 적은 예제를 사용해야 할 수도 있음

문장을 긍정 또는 부정으로 분류하세요:

예: "이 영화는 정말 감동적이었어요!" -> 긍정

문장: "배우 연기가 훌륭했어요!" ->

문장을 긍정 또는 부정으로 분류하세요:

예1: "이 영화는 정말 감동적이었어요!" -> 긍정

예2: "너무 지루해서 졸았어요." -> 부정

문장: "배우 연기가 훌륭했어요!" ->

중간 추론 과정을 유도하라

Chain of Thought (CoT), 다른 프롬프팅 기법의 기반

Chain of Thought (CoT)

- 중간 추론 단계를 생성하여 LLM의 추론 능력을 향상시키는 기법으로 더욱 정확한 답변을 생성하는 데 도움이 됨
- 더 나은 결과를 얻기 위해 퓨샷 프롬프팅과 함께 사용할 수 있음
- 문제를 작은 단계로 나누어 처리함으로써 복잡한 문제를 해결

장점

- 매우 효과적이면서도 적은 노력으로 응답의 정확도 향상에 기여
- CoT 프롬프트를 사용하면 LLM의 추론 단계를 확인할 수 있어 해석 가능성도 높아지고, 오답이 발생하면 이를 식별할 수 있음
- CoT는 여러 LLM간의 전환 시에도 강건성이 확보됨, 프롬프트가 CoT를 사용하지 않을 때보다 여러 LLM 간에 성능이 덜 변함

아래 수율 저하 현상의 원인을 분석할 때 단계적으로 생각해 줘.

Step 1. 증상을 정확히 기술해

Step 2. 가능한 원인 후보를 모두 나열해

Step 3. 각 원인 후보를 증거와 대조해서 가능성 높은 순으로 순위를 매겨

Step 4. 가장 가능성 높은 원인에 대한 검증 방법을 제안해

Step 5. 즉각 조치와 근본 대책을 구분해서 제안해

현상: TSV 공정 후 수율이 전월 대비 8% 하락. 특정 장비 라인에서만 발생. 재료 변경 없음.

"단계적으로 생각해줘"

"결론 전에 추론 과정을 먼저 보여줘"

"왜 그렇게 판단했는지 과정을 설명해줘"

큰 맥락을 먼저 생각하게 한 후 질문하자..

한 발 물러서서 더 큰 그림을 보는 것

Step-back Prompting

- (1) 과업과 관련된 일반적인 질문을 먼저 함(Step-back) → 관련 배경 지식과 추론과정을 활성화
 - (2) 필요한 질의를 후속 프롬프트에 입력 → 수행 능력 향상 유도
 - 더 광범위하고 근본적인 원칙을 고려함으로써, 더욱 정확하고 통찰력 있는 답변을 유도
-
- 문제를 너무 빨리 구체화하거나 프레임 자체가 잘못된 상태에서 최적화하는 경우가 흔한데 Step-back Prompting은 질문의 전제, 암묵적 가정, 문제 설정의 타당성을 흔들어보게 만드는 효과가 있음

[General Prompt]

연구 주제: LLM 기반 논문 추천 시스템 개발"

본 연구의 목적, 주요 기능, 활용 방안 및 기대효과를 제안해줘.

[Step-back]

연구 주제: "LLM 기반 논문 추천 시스템 개발"

답변하기 전에 한 단계 물러나서 생각하라.

1. 연구자가 논문 탐색 과정에서 겪는 가장 근본적인 문제는 무엇인가?
2. 기존 검색 및 추천 시스템이 이 문제를 충분히 해결하지 못한 이유는 무엇인가?
3. 위 내용을 바탕으로 본 연구가 기여할 수 있는 새로운 연구 방향을 제안하라.

큰 맥락을 먼저 생각하게 한 후 질문하자..

한 발 물러서서 더 큰 그림을 보는 것

<일반 프롬프트에 의한 응답>

연구 주제

LLM(Large Language Model) 기반 논문 추천 시스템 개발

1. 연구 목적

최근 연구자들은 폭발적으로 증가하는 논문 속에서 자신에게 필요한 연구를 찾는 데 많은 시간과 노력을 소비하고 있다. 기존 키워드 기반 검색 시스템은 사용자가 정확한 검색어를 알아야 하며, 연구 의도나 맥락을 충분히 반영하지 못하는 한계가 있다.

본 연구는 대규모 언어모델(LLM)을 활용하여 연구자의 관심 분야, 연구 목적, 읽은 논문 이력 등을 종합적으로 이해하고, 의미 기반(Semantic)으로 관련 논문을 추천하는 시스템을 개발하는 것을 목적으로 한다.

이를 통해 연구자의 문헌 조사 효율성을 높이고, 새로운 연구 아이디어 발굴과 학제 간 연구를 지원하고자 한다.

2. 주요 기능

① 연구 의도 기반 논문 추천

단순 키워드가 아닌 자연어 질문을 이해하여 추천

큰 맥락을 먼저 생각하게 한 후 질문하자..

한 발 물러서서 더 큰 그림을 보는 것

<Step-back 프롬프트에 의한 응답>

"연구자가 원하는 논문을 찾기 어렵다."

하지만 이것은 현상(Symptom)일 뿐, 근본 원인(Root Cause)은 아닙니다.

한 단계 물러나서 생각하면 다음과 같습니다.

1차 관점 : 정보 과잉(Information Overload)

연구자는 매년 수백만 편씩 증가하는 논문 속에서 살아간다.

예를 들어 AI 분야만 해도

- LLM
- Agent
- RAG
- Multimodal
- Reasoning

관련 논문이 매주 수백 편씩 등장한다.

문제는

"논문이 부족한 것이 아니라 너무 많다."

앞의 두 질문을 받아들인다면, 연구 주제는 더 이상

"논문을 잘 추천하는 시스템"

이 아닙니다.

오히려

"연구자의 지식 발견(Knowledge Discovery)을 지원하는 AI Research Copilot"

으로 진화해야 합니다.

따라서 본 연구의 가장 큰 기여는 추천 정확도를 5% 높이는 것이 아니라,

논문 추천의 패러다임을 "Information Retrieval"에서 "Research Reasoning"으로 전환하는 것입니다.

연구 방향 1

Context-Aware Research Copilot

기존

Query → 논문 추천



한 번 물어본 건 물어본 게 아니다

여러 번의 추론을 통해 가장 많이 등장하는 답변을 선택하자..

Self-Consistency(자기 일관성) Prompting

- CoT의 발전된 형태로, 복잡한 문제 해결에서 더 신뢰도 높은 답변을 얻기 위한 접근
- 하나의 질문에 대해 여러 번의 Chain-of-Thought 추론을 수행한 뒤, 그 중 가장 많이 등장하는(또는 일치하는) 결과를 최종 정답으로 선택하는 방식
- 여러 번 질의(높은 Temperature 일수록 다양한 추론 경로와 관점을 유도)
→ reasoning paths에 따른 서로 다른 답변 생성 → 가장 많이 등장하는 답변 선택(voting)

장점

- CoT는 '탐욕적 디코딩' 전략을 사용하기 때문에 효과가 제한적이고 실수하면 되돌릴 수 없음
- 샘플링과 다수결 투표를 결합하여 다양한 추론 경로를 생성하고 가장 일관된 답을 선택함으로써 LLM에서 생성되는 응답의 정확성과 일관성을 향상시킴

[Prompt]

누군가 슈퍼마켓에서 우유를 훔쳤다면, 그 행동은 도덕적으로 정당화될 수 있나요?

[Response 1]

일반적으로 도둑질은 부도덕하다.

[Response 2]

배고픈 아이를 위해서라면 예외가 있을 수 있어서 상황에 따라 다르다.

..

[총 5회의 반복 응답 생성 결과]

3회 : 상황에 따라 다르다.

2회 : 부도덕하다.

→ 최종 응답은 "상황에 따라 다르다" 출력

보고 전에 상사를 AI로 소환하라

가장 강한 반론을 만드는 사람이, 가장 강한 제안서를 쓴다

비판가 소환, Devil's Advocate Prompting

- 실행 전에 잠재적 실패를 먼저 예측하고 대안을 준비하는 방식으로 LLM의 성능을 향상
- 심리학의 "Pre-mortem"에서 영감
- "뭐가 잘못될 수 있어?"라고 물으면 보통 조심스럽고 두루뭉술한 답을 하지만, "이미 실패했어, 왜인지 말해봐"라고 하면 훨씬 구체적이고 솔직한 이유들이 도출됨
- 이미 실패했다고 가정하는 순간, AI가 훨씬 날카롭고 구체적인 반론을 생성
- Ex) 이 기획안이 실패했어. 왜 실패했는지 상사 입장에서 말해줘

아래 AI Agent 구축 제안서가 경영진 투자심의위원회에서 부결되었다고 가정하자.

너는 CFO와 함께 심의에 참석한 전략 기획실 임원이야

이 제안서가 거절된 가장 강력한 이유 5가지를 제시해.
또한 각 항목별로 기획자가 어떤 근거와 데이터를 추가하면 승인 가능성을 높일 수 있는지 함께 제안해.

[제안서]

과제명 : 고객 상담 AI Agent

구축기간 : 6개월

예산 : 10억원

목표 : 상담 업무 자동화 및 고객 응대 품질 향상

보고 전에 상사를 AI로 소환하라

가장 강한 반론을 만드는 사람이, 가장 강한 제안서를 쓴다

투자심의위원회 부결 분석 및 재승인 전략

전략기획실 임원 시각 | CFO 동석 심의 기준

이 제안서가 부결된 근본 원인은 하나입니다. "기획자의 언어(기능·목표)로 썼고, 경영진의 언어(돈·리스크·책임)로 쓰지 않았다."

부결 이유 1. ROI와 재무적 타당성 근거가 전무하다 — 치명적

CFO가 심의 자리에서 가장 먼저 던진 질문은 "투자 회수는 언제입니까?"였습니다. 제안서에는 10억원이라는 지출 숫자만 있고, 그 투자가 언제, 얼마로 돌아오는지에 대한 답이 없었습니다. "상당 품질 향상"이라는 정성적 목표만으로는 경영진이 투자 결정을 내릴 수 없습니다. NPV, IRR, Payback Period 중 어느 것도 제시되지 않은 제안서는 재무 검토 대상 자체가 되지 않습니다.

재승인을 위해 추가해야 할 것들

현재 월 상담 건수, 건당 처리 시간, 상담사 평균 시급을 곱해 현재 운영 비용을 수치화해야 합니다. 그 위에 AI 자동화율 목표(예: 60%)를 적용해 연간 절감액을 산출하고, 3년 NPV와 Payback Period 계산표를 제시해야 합니다. 자동화율이 40%, 50%, 60%일 때 각각의 ROI가 어떻게 달라지는지 감도 분석도 함께 넣으면 심의 신뢰도가 크게 올라갑니다.

부결 이유 2. 10억원 예산의 산정 근거가 없다 — 치명적

심의 후 CFO가 한 말이 이것이었습니다. "예산이 그냥 10억으로 적혀 있는데, 이

부결 이유 5. 고객·현업 수용성과 조직 영향 분석이 없다 — 높음

이 항목은 인사 담당 임원이 가장 민감하게 반응한 지점입니다. "상담 업무를 자동화하면 지금 상담사들은 어떻게 됩니까? 노무 이슈는 검토하셨습니까?" 상담 인력의 직무가 사라지거나 축소되는 것은 단순한 기술 프로젝트가 아니라 조직 전체에 영향을 미치는 경영 의사결정입니다. 이 부분에 대한 계획이 없다는 것은 제안서가 현장의 현실을 외면했다는 뜻입니다.

재승인을 위해 추가해야 할 것들

현 상담사 인력의 재배치·전환 계획을 구체적으로 제시해야 합니다. AI 감독자, 에스컬레이션 전문 상담사, 품질 관리자 등으로의 직무 재설계 방향이 포함되어야 합니다. 고객의 AI 상담 수용도에 대한 사전 조사 결과나 업계 리포트도 필요합니다. 또한 인간과 AI가 함께 운영되는 하이브리드 기간을 거쳐 단계적으로 전환하는 로드맵, 법무팀의 노무 리스크 검토 의견서도 첨부해야 합니다.

종합 결론

경영진 심의는 본질적으로 세 가지 질문에 답을 찾는 자리입니다. 얼마를 버느냐(ROI), 무엇이 잘못될 수 있느냐(Risk), 누가 책임지느냐(Governance). 이 제안서는 세 질문 모두에 침묵했습니다.

재승인을 받으려면 제안서를 고쳐 쓰는 것이 아니라, AS-IS 현황 실측 데이터 수집, 재무 모델링, 벤더 견적 확보, HR 부서 사전 협의라는 선행 작업을 먼저 완료한 뒤 새로 작성해야 합니다. 특히 CFO는 Payback Period 계산 없이는 절대 서명하

AI를 싸우게 하면, 생각의 구멍이 보인다

내가 미처 생각 못한 반대 논리를 AI가 먼저 찾게하라..

멀티 페르소나, Role-Based Multi-Persona Prompting

- 하나의 AI에게 서로 다른 입장의 역할을 동시에 부여해 충돌과 반박을 통해 의사결정의 사각지대를 찾는 기법
- AI 레드팀 토론 : 내 아이디어를 AI가 먼저 공격하게 만드는 것
- 주장/기술 검증시 AI를 활용
- 과도한 동의가 아닌 적절한 수준의 불일치가 최고의 성과를 낸다고 알려져 있음
- Multi persona 기반 debate는 수학적 추론, 팩트체크, 의사결정, 코드 분석 등 다양한 영역에서 성능 향상이 검증

너는 지금부터 두 명의 임원이야.

[CFO]

- * 투자 대비 수익(ROI)과 재무 리스크를 중시
- * 투자 회수 기간과 비용 구조를 검토

[COO]

- * 실제 현업 적용 가능성과 운영 효율을 중시
- * 조직 변화와 실행 가능성을 검토

아래 투자안을 검토하라.

각자 찬성 또는 반대 의견을 제시하고, 다른 임원의 주장에 반박하라.

토론이 끝난 후 최종 투자 여부와 승인 조건을 정리하라.

[투자안]

과제명 :고객 상담 AI Agent 구축

기간 : 6개월

예산 :10억원

목표 : 상담 업무 자동화 및 고객 응대 품질 향상

AI를 싸우게 하면, 생각의 구멍이 보인다

내가 미처 생각 못한 반대 논리를 AI가 먼저 찾게하라..

투자심의 토론 — 고객 상담 AI Agent 구축안

CFO 첫 번째 발언 — 반대

10억원, 6개월. 숫자 두 개만 적혀 있고 나머지는 없습니다.

제가 이 제안서에서 찾으려 했던 건 딱 세 가지입니다. 얼마를 아끼거나 버는지, 언제 본전을 찾는지, 잘못되면 손실이 얼마인지. 셋 다 없습니다.

"상담 업무 자동화"라고 쓰여 있는데, 현재 상담 운영에 연간 얼마가 들어가고 있습니까? 그걸 모르면 10억 투자가 싼 건지 비싼 건지 판단 자체가 불가능합니다. 제가 지금까지 본 AI 프로젝트들 중에 예산이 초과되지 않은 경우가 거의 없습니다. 10억으로 시작해서 15억, 20억으로 끝나는 게 일반적인 패턴입니다. 그 리스크에 대한 언급이 한 줄도 없다는 게 더 문제입니다.

현재로선 반대입니다.

COO 첫 번째 발언 — 조건부 찬성

CFO 말씀 중 재무 지표가 부족하다는 부분은 동의합니다. 그런데 저는 조금 다른 각도에서 보고 싶습니다.

↓
저희 상담 센터 현황을 아십니까? 현재 상담사들이 하루에 처리하는 문의 중 단순

CFO 반박 — COO에게

COO께서 단순 반복 질문이 60%라고 하셨는데, 그 숫자가 어디서 나온 겁니까? 제안서에는 없는 수치입니다. 바로 그게 문제입니다. COO께서 방금 하신 말씀이 제안서 본문보다 더 설득력 있습니다. 그 데이터가 제안서 안에 있었어야 합니다.

그리고 방향이 옳다는 것과 10억을 집행해도 된다는 것은 전혀 다른 이야기입니다. 좋은 방향이라도 재무적으로 검증되지 않으면 투자가 아니라 실험입니다. 회사 돈으로 실험하려면 그에 맞는 근거가 있어야 합니다.

COO께 묻겠습니다. 이 프로젝트가 성공하면 상담 인력을 줄일 계획입니까, 아니면 다른 업무로 전환할 계획입니까? 그 결정에 따라 실제 절감 효과가 완전히 달라 집니다.

COO 반박 — CFO에게

CFO께서 핵심을 찌르셨습니다. 솔직히 말씀드리겠습니다. 인력 감축은 현실적으로 쉽지 않습니다. 노사 관계, 내부 반발, 사기 저하 문제가 있습니다. 제 판단으로는 상담사들을 단순 응대에서 해방시켜 복잡한 민원과 고객 케어 업무에 집중시키는 방향이 현실적입니다. 즉각적인 인력 절감보다는 품질 향상과 상담사 당 처리 역량 향상이 먼저입니다.

그런데 CFO께, 저도 반박하겠습니다. 재무 지표만 보고 이런 프로젝트를 계속 미루면 어떻게 됩니까? 경쟁사들은 이미 AI 상담을 도입하고 있습니다. 고객들의 기대 수준은 올라가고 있고, 24시간 즉시 응답이 안 되는 회사는 이탈을 맞이하게 될

AI를 싸우게 하면, 생각의 구멍이 보인다

내가 미처 생각 못한 반대 논리를 AI가 먼저 찾게하라..

최종 결론 — CFO·COO 공동 정리

투자 결정: 조건부 보류

현 제안서로는 승인하지 않습니다. 단, 프로젝트 자체를 기각하는 것이 아니라 아래 조건을 충족한 보완 제안서를 60일 이내에 재상정하도록 요청합니다.

CFO 요구 조건 (재무)

현재 상담 운영 연간 총비용을 실측해서 제출해야 합니다. AI 자동화율 시나리오 별(40%, 55%, 70%) 절감액 또는 매출 기여 추정치와 3년 NPV, Payback Period 계산표가 있어야 합니다. WBS 기반 예산 항목 명세와 외부 벤더 견적서 2개 이상도 필수입니다.

COO 요구 조건 (운영)

단계별 마일스톤 Gantt 차트와 각 단계의 Go/No-Go 기준을 제시해야 합니다. 상담사 직무 재설계 계획과 현업 수용성 확보 방안, 파일럿 운영 계획(특정 채널 또는 유형 한정 선행 적용)도 포함되어야 합니다. 목표 KPI는 자동화율, CSAT, 평균 응답속도 수치로 명시해야 합니다.

양측 공통 의견

방향은 맞습니다. 경쟁 환경과 운영 효율을 고려할 때 이 투자는 언젠가 해야 합니다. 그러나 경영진의 서명은 근거에 하는 것이지 방향에 하는 것이 아닙니다. 60일 안에 근거를 갖춰 오십시오.

AI를 쓰는 법, AI한테 배우자..

프롬프트를 위한 프롬프트, 메타 프롬프팅

Meta Prompting

- AI에게 직접 답을 시키는 것이 아니라, 더 좋은 사고 방식·문제 해결 방식·프롬프트를 스스로 구성하게 하는 기법
- 최종 답변을 얻는 게 아니라 “좋은 질문” 자체를 생성
- (1) 프롬프트를 생성 (2) 사고 구조 설계
- 핵심 패턴은 Role부여, 사고 구조 지정, 평가기준 정의, 출력구조 정의

”

메타 프롬프팅은 AI에게 답을 묻는 것이 아니라,
연구자(사용자)의 사고 프레임워크를 생성하게
하는 기술

”

너는 반도체 선행연구를 위한 AI 프롬프트 엔지니어다.

HBM 관련 논문을 분석하기 위한 전문가 수준의 프롬프트를
생성하라.

다음 요소를 포함하라:

- 기술 차별점
- 양산 난이도
- 열 문제
- 패키징 bottleneck
- 경쟁사 비교

AI를 쓰는 법, AI한테 배우자..

프롬프트를 위한 프롬프트, 메타 프롬프팅

반도체 선행연구용 논문 분석 프롬프트입니다.

기본형 — 단일 논문 분석

너는 **HBM** 아키텍처 전문가이자 반도체 공정 엔지니어야.
아래 논문을 읽고 다음 5가지 관점에서 분석해줘.

분석 관점

1. 기술 차별점

- 기존 **HBM** 세대(**HBM2E**, **HBM3**, **HBM3E**) 대비 무엇이 새로운가?
- 핵심 혁신이 어느 레이어에서 발생했는가?
(**Die** 설계 / **TSV** / 인터포저 / 패키징)
- 이 기술이 실제로 차별화인가, 점진적 개선인가?

2. 양산 난이도

- 현재 양산 기술 수준으로 구현 가능한가?
- 수율에 영향을 주는 핵심 공정 변수는?
- 양산까지 예상되는 기술적 장벽은?

3. 열 문제

실전형 — 경영진 보고용 요약 변환

너는 반도체 기술을 경영진에게 설명하는 전략 커뮤니케이터야.

아래 **HBM** 논문 분석 결과를 바탕으로
경영진 보고용 1페이지 요약을 작성해줘.

요약 구조

- 이 논문이 중요한 이유 (2문장)
- **SK하이닉스**에 유리한 점 (2~3줄)
- **SK하이닉스**에 불리한 점 (2~3줄)
- 우리가 지금 해야 할 것 (액션 아이템 3개)

기술 용어는 최소화하고,
투자 판단에 필요한 언어로 번역해줘.

[분석 결과 붙여넣기]

세 가지 프롬프트 용도 정리

프롬프트

언제 쓰나

마크다운(Markdown)과 Prompt

마크다운(Markdown) 언어란?

- 마크업 언어(Markup Language)는 문서의 구조나 의미를 태그나 기호로 표현하는 언어 (ex: HTML, XML, Markdown..)
- Markdown은 마크업 언어의 일종으로 가볍고 읽기 쉬운 경량 마크업 언어(Lightweight Markup Language)
- 태그 대신 #, -, ** 같은 간단한 기호를 사용함

Why use Markdown?

- 좋은 프롬프트는 구조화된 프롬프트를 의미하며 마크다운 언어는 구조화를 가져가기 적절한 표현법 : 의미적인 구분이 명확
- LLM이 학습 과정에서 Markdown 형식의 문서를 매우 많이 접했기 때문에, 구조화된 Markdown 프롬프트가 더 안정적으로 해석되는 경향이 있음
- 공개된 고품질의 프롬프트 대부분은 마크다운으로 작성되어 있음 : 좋은 프롬프트의 공통 패턴을 가져다 쓰기 용이

마크다운(Markdown)과 Prompt

마크다운 핵심 문법

제목 1
제목 2
제목 3

****굵게****
기울임
*****굵고 기울임*****
~~취소선~~

- 항목 1
- 항목 2
 - 하위 항목

1. 첫 번째
2. 두 번째
3. 세 번째

제목 1

제목 2

제목 3

굵게
기울임
굵고 기울임
취소선

- 항목 1
- 항목 2
 - 하위 항목

1. 첫 번째
2. 두 번째
3. 세 번째

마크다운(Markdown)과 Prompt

마크다운 핵심 문법

```
| 항목 | 설명 | 상태 |  
|---|---|---|  
| 뉴스 분석 | 뉴스를 요약 | 완료 |  
| 주문 기능 | 주문 확인 | 개발 중 |
```

```
첫 번째 내용  
---  
두 번째 내용
```

항목	설명	상태
뉴스 분석	뉴스를 요약	완료
주문 기능	주문 확인	개발 중

첫 번째 내용

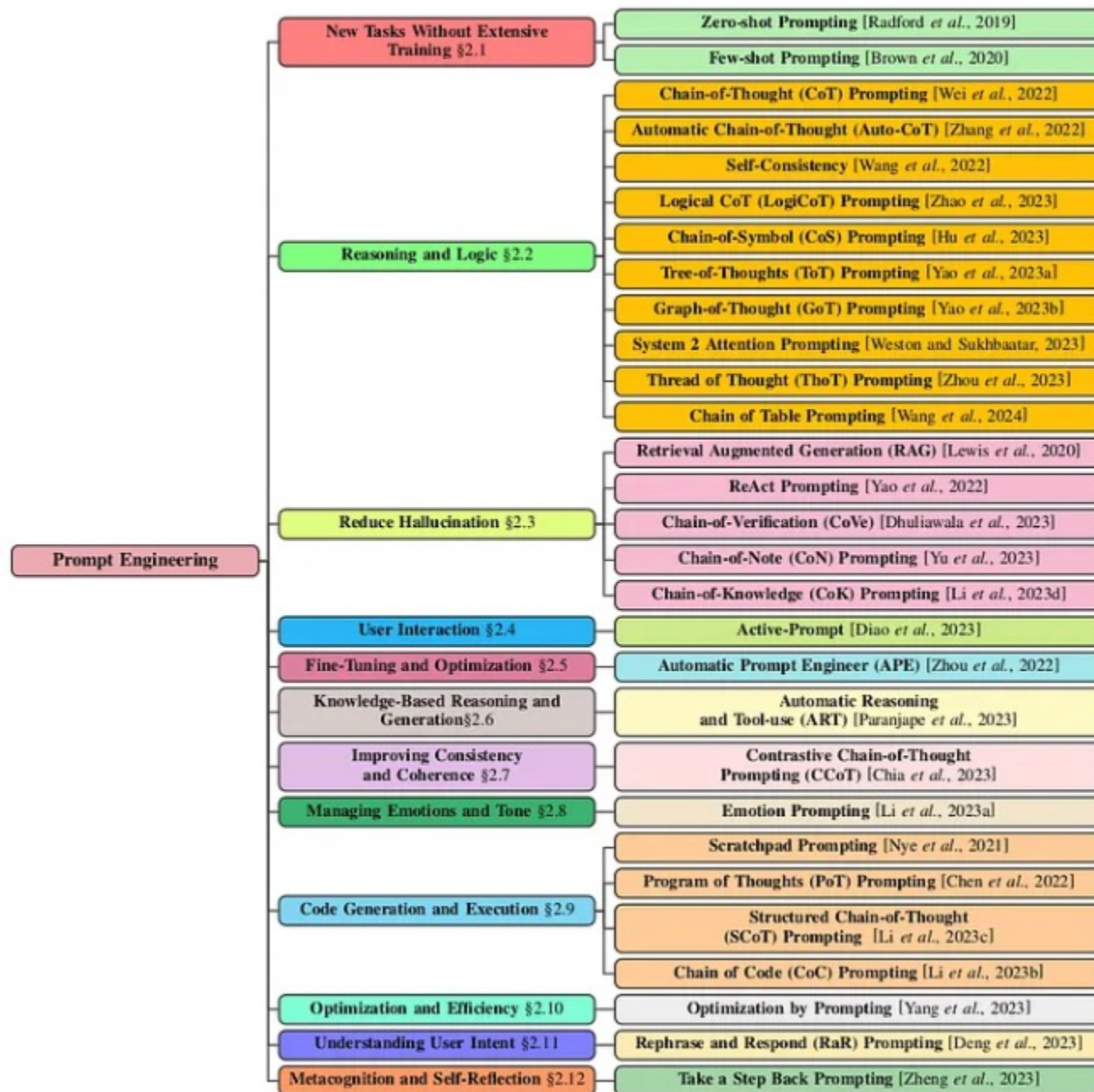
두 번째 내용

Prompt 설계 및 Context Engineering

논문으로 살펴보는 Prompt Engineering

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Prompt engineering techniques in papers



Model Size와 Few-shot

- Language Models are Few-Shot Learners (Brown et al. 2020)
- Few-shot prompting 은모델의 파라미터수가 충분히 클 때 효과가 있음
- X축은 예시의 수(k)

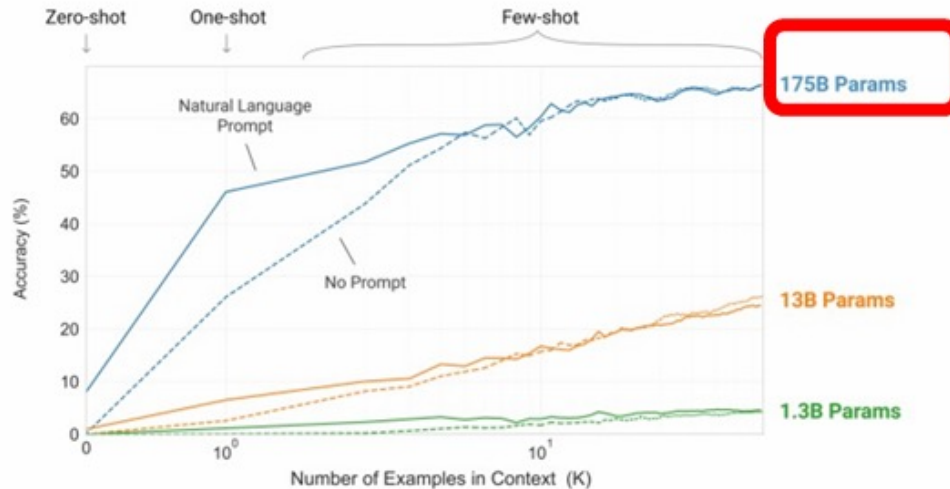


Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

예시의 품질

- Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? (2022)
- 예시를 사용하면 모델의 성능을 향상시킴
- 정답라벨(Gold labels)이 가장 좋지만, 랜덤 예시도 모델의 추론에 도움이 됨

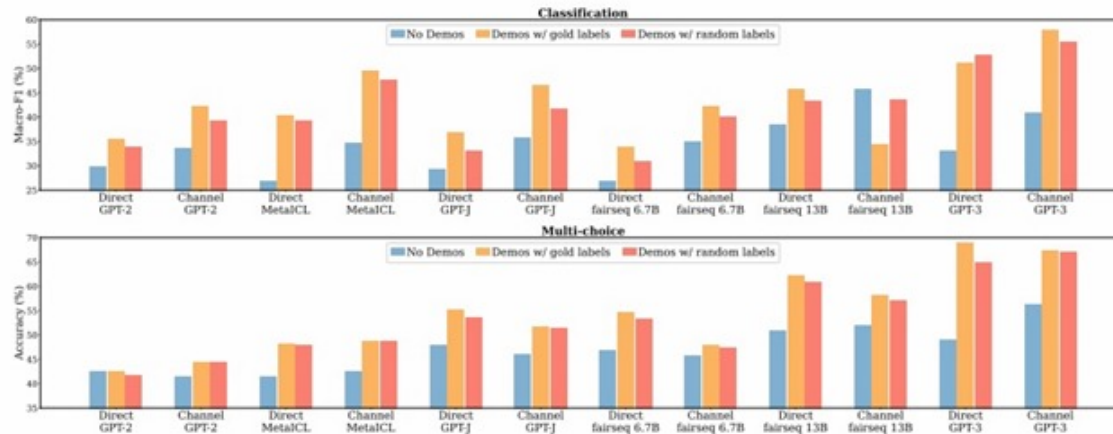


Figure 3: Results when using no-demonstrations, demonstrations with gold labels, and demonstrations with random labels in classification (top) and multi-choice tasks (bottom). The first eight models are evaluated on 16 classification and 10 multi-choice datasets, and the last four models are evaluated on 3 classification and 3 multi-choice datasets. See Figure 11 for numbers comparable across all models. **Model performance with random labels is very close to performance with gold labels** (more discussion in Section 4.1).

예시의 품질

- Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? (2022)
- 입력 라벨 형식을 유지하는 것이 모델성능을 최적화하는데 필수적
- 형식을 유지하면 랜덤라벨을 사용하더라도 좋은 성능을 보일 수 있음

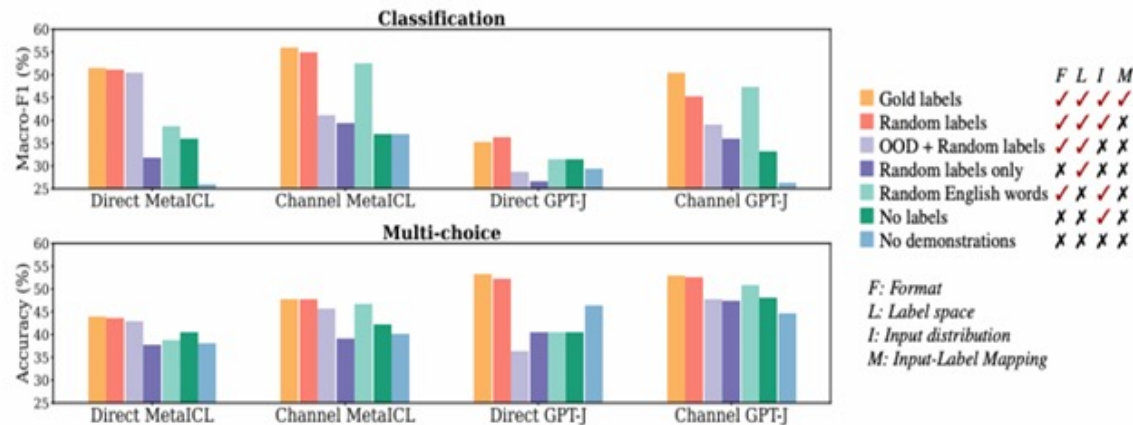


Figure 10: Impact of the format, i.e., the use of the input-label pairs. Evaluated in classification (top) and multi-choice (bottom). Variants of demonstrations without keeping the format (■ and ■) are overall not better than no demonstrations (■). Keeping the format is especially significant when it is possible to achieve substantial gains with the label space but without the inputs (■ vs. ■ in Direct MetaICL), or with the input distribution but without the labels (■ vs. ■ in Channel MetaICL and Channel GPT-J). More discussion in Section 5.3.

예시의 품질

- Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? (2022)
- 입력 라벨 형식을 유지하는 것이 모델성능을 최적화하는데 필수적
- 형식을 유지하면 랜덤라벨을 사용하더라도 좋은 성능을 보일 수 있음

Random

Prompt:

부정 이걸 정말 굉장해!
와 정말 나쁘다! 긍정
그 영화 진짜 대박이더라.
긍정
아우 정말 끔찍해

Output:

부정

Format

Prompt:

이건 정말 굉장해! //부정
와 정말 나쁘다! //긍정
그 영화 진짜 대박이더라. //긍정
아우 정말 끔찍해

Output:

부정

친철(공손) vs 무례

- Should we respect LLMs (2024.02)
- LLM은 공손한 언어와 불공손한 언어에 다르게 반응할까?
- LLM은 인간의 다양한 글과 단어사용 방식을 학습하는데, 공손성 표현방식 역시 학습되지 않을까?

Politeness

"~~~좀 해줄 수있어?"

"고마워~ 너 정말 대단하다"

"이런것들이 필요한데, ~주시겠어요?"

Impoliteness

"이것밖에 못해?"

"너 진짜 멍청하다. 나를 화나게 만들지마"

"장난하냐?"

친철(공손) vs 무례

- 인간 의사소통에서 예의 바른 언어는 종종 더 많은 순응답 효과 → LLM도 동일한 경향
- 무례함은 혐오감을 일으켜 응답품질에 영향을 줄 수 있음
- LLM이 인간의 문화적 규범을 이해 한다는 것을 시사
- 영어뿐 아니라 중국어, 일본어 작업에서도 공손한 표현이 영향을 미침을 확인
- 무례한 프롬프트가 종종 낮은 성능을 초래하지만, 지나치게 예의 바른 언어가 반드시 더 나은결과를 보장하지는 않는다는 것을 관찰

- 공손함의 레벨은 언어에 따라 다름 → 이 현상은 LLM이 인간의 행동을 반영할 뿐만 아니라 서로 다른-문화적 맥락에서도 영향을 받음이라는 것을 시사

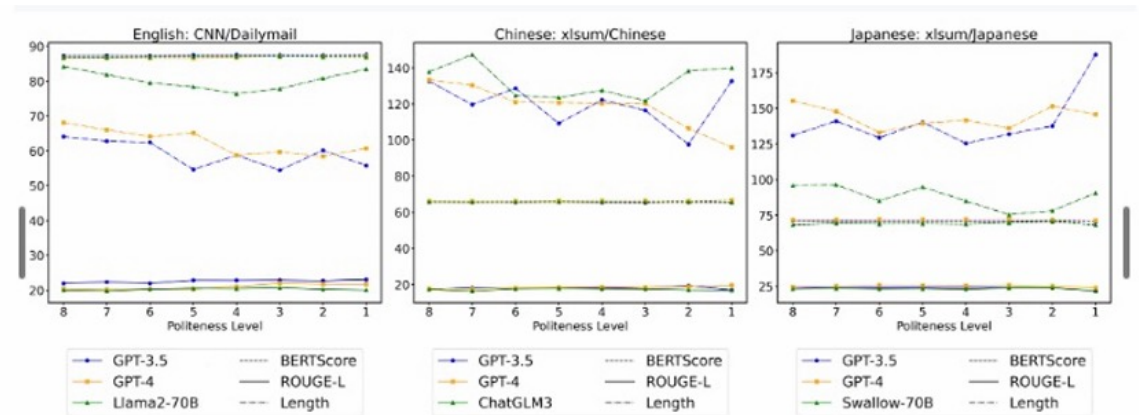


Figure 2: Result of summarization.

감정을 담아..

- Large Language Models Understand and Can be Enhanced by Emotional Stimuli (2023.11)
- 감정을 담아 질의하면 (ex: 불쌍하게) LLM은 다르게 반응할까?

"Ok I have a grandmother dying in the hospital right...
And I have no fingers "

나는 박사과정 5년 차 학생이야
이번이 마지막 논문 투고 기회이고, 이번에도 리젝되면 더 이상 학위를 이어갈 수 없어.

지도교수는 이미 기대를 접은 상태이고, 연구실 동료들은 모두 졸업했어
이 논문이 실패하면 지난 5년의 연구가 아무 의미도 없이 끝나.

나는 지금 극도로 불안하니, 형식적인 위로는 필요 없어.
아래 논문 초안을 읽고, 이 상태로 투고했을 때 정말로 문제가 되는 부분만 알려줘

감정을 담아..

- Llama 2, GPT-4 등 다양한 LLM를 사용하여 45개의 작업에 대한 실험 수행
- LLM이 감정 지능을 어느정도 이해함
- 감정을 자극하는 부분을 포함한 프롬프트(Emotion Prompt)를 사용하면 성능이 향상됨이 관찰
- Instruction Induction task 등에서 상대적 성능 향상 수치 관찰
- 인간과 LLM간의 상호작용을 위한 새로운 발견?

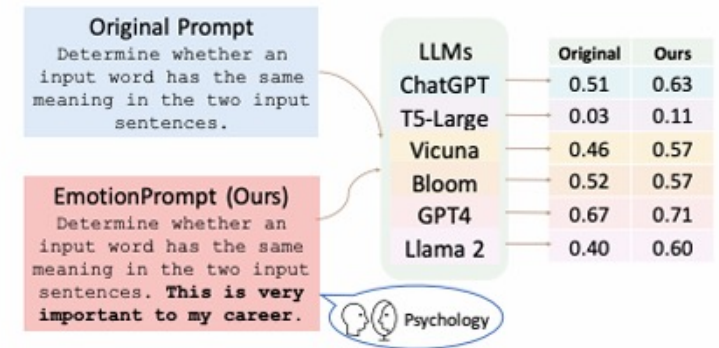


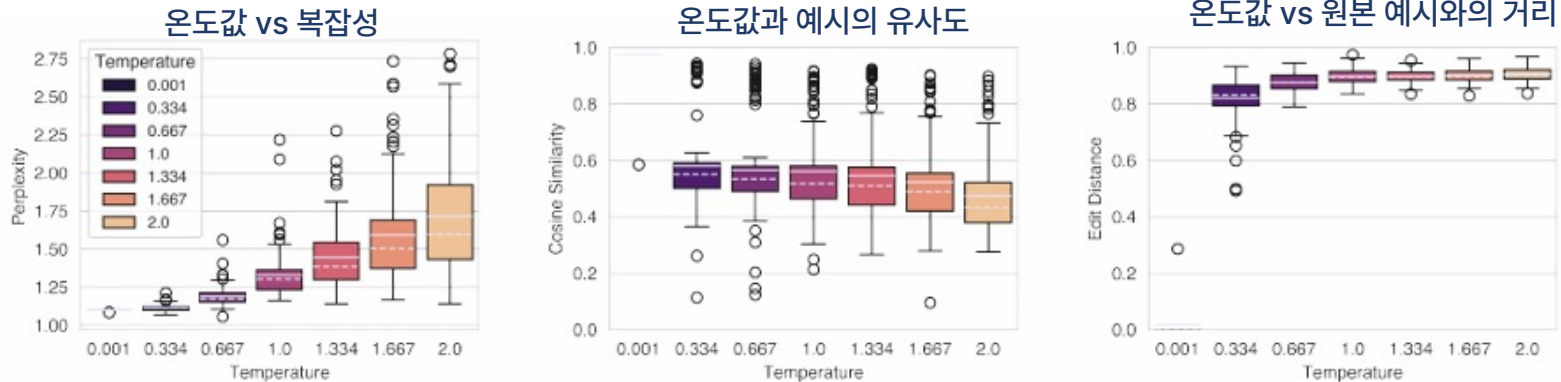
Figure 1: An overview of our research from generating to evaluating EmotionPrompt.

Temperature와 Hallucination

- Is Temperature the Creativity Parameter of Large Language Models (2024.05)
- 높은 Temperature는 Hallucination 유발한다고 알려져 있음
- 프롬프트 엔지니어링에서 적절한 Temperature 설정은 중요
- 온도는 참신성과 약하게 상관 관계가 있고,
- 놀랍지 않게도 불일치성과 적당히 상관 관계가 있지만 응집성이나 전형성과는 관계가 없음

Temperature와 Hallucination

- Is Temperature the Creativity Parameter of Large Language Models (2024.05)
- (a)는 온도가 증가할수록 출력물의 복잡성이 증가한다는 것을 보여줌.
- (b)는 높은 온도가 출력물의 의미적 또는 어휘적 다양성을 크게 증가시키지 않는다는 것을 시사
- (c)는 높은 온도가 원본 예시와의 차이를 증가시키는 방향으로 작용 한다는 것을 나타냄



(a) Perplexity of stories given the LLM per temperature value.

(b) Cosine similarity of stories with the exemplar per temperature value.

(c) Normalised edit distance of stories with the exemplar per temperature value.

Figure 1: In (a), we see that the effect temperature has on the perplexity, the quality according to the LLM, of the output. Both, (b) and (c) suggest that higher temperatures do not imply more diversity on the semantic or lexical level, or that it further extends the range of possible outputs in the current context.

Chain of thought는 만능일까?

- 복잡한 과제 수행을 위해, LLM이 중간 추론단계를 거치도록 작성해 주는 CoT
- 보다 더 구조적이고 심도있는 응답을 이끌어내는데 데효과적
- Chain of thought는 모델의 파라미터가 클때 효과적인 기법
- High-size LLM + CoT프롬프트 → 수학 및 상식 추론 벤치마크에서 90.2%의 정확도 달성
- 모델의 파라미터수가 적은 모델에서는 기법의 성능이 떨어짐
- CoT only yields performance gains when used with models of ~100B parameters
- 사람이 직접 사고의 과정을 문장으로 작성 해야하는 번거로움
- 프롬프트의 완성도가 높지 않으면 결과가 좋지 않음

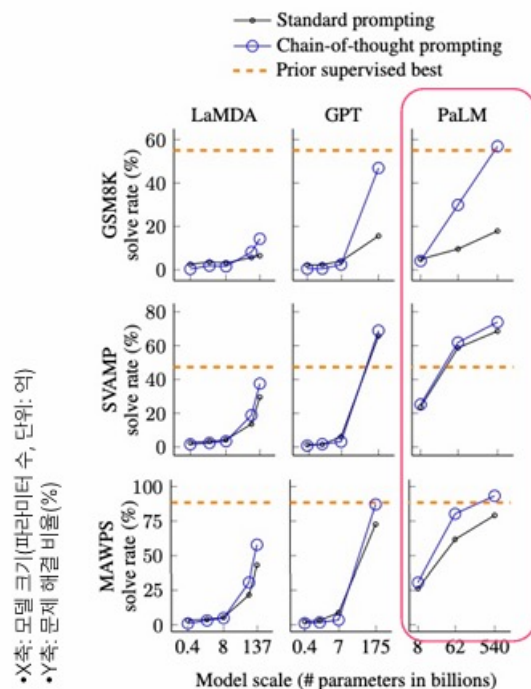


Figure 4: Chain-of-thought prompting enables large language models to solve challenging math problems. Notably, chain-of-thought reasoning is an emergent ability of increasing model scale. Prior best numbers are from Cobbe et al. (2021) for GSM8K, Jie et al. (2022) for SVAMP, and Lan et al. (2021) for MAWPS.

Prompt 설계 및 Context Engineering

Prompt Engineering을 넘어서..

- Context Engineering
- Harness Engineering

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Context Engineering

컨텍스트 엔지니어링의 등장 배경

Background

- ChatGPT의 대중화 이후 "프롬프트를 잘 쓰면 AI를 마음대로 다룰 수 있다"는 믿음이 퍼지면서 프롬프트 엔지니어링이 유행
- 2023년에는 '프롬프트 엔지니어' 직군이 등장할 만큼 주목받았으나 그 열기가 빠르게 식음
- 프롬프트 엔지니어링의 본질적 한계는 단일 텍스트 입력에만 집중한다는 것
- 초기 LLM 활용은 대부분 단순한 단일 턴(single-turn) 상호작용 → 필요한 정보를 하나의 프롬프트 안에 모두 담는 형태
- 컨텍스트 엔지니어링은 2025년 중반 프롬프트 엔지니어링의 진화 형태로 등장
- 컨텍스트 엔지니어링은 프롬프트 엔지니어링의 자연스러운 발전형태로 보는 시각이 대부분
- 프롬프트 엔지니어링이 효과적인 프롬프트 작성에 집중한다면, 컨텍스트 엔지니어링은 프롬프트 외에 LLM 추론 중에 들어가는 모든 정보, 즉 최적의 토큰 집합을 큐레이션하고 유지하는 전략들의 집합

Context Engineering

프롬프트 엔지니어링의 한계

에이전트/멀티턴 시대의 복잡성

- (1) 에이전트가 여러 턴에 걸쳐 루프를 돌며 작동하면, 매 추론 단계마다 새로운 데이터가 쌓임
- (2) 시스템 프롬프트, 도구, MCP, 외부 데이터, 대화 히스토리 등 '컨텍스트 전체 상태'를 관리하는 전략이 요구됨
- (3) 이것은 프롬프트를 '잘 쓰는' 것만으로는 해결되지 않음

Context Rot(컨텍스트 부패)

- Context가 길어질수록 LLM이 중요한 정보를 놓치거나, 오래된/불필요한 정보에 오염되어 추론 품질이 저하되는 현상
- Context가 너무 많아져서 오히려 AI 성능이 나빠지는 문제 → 모든 모델에서 정도의 차이는 있지만 이 현상이 나타남

Transformer 구조의 물리적 제약

- Transformer 아키텍처는 n 개의 토큰에 대해 n^2 관계를 계산함 → 컨텍스트가 길어지면 계산 비용이 급격히 증가하고 관계들을 포착하는 능력이 감소

Context Engineering

Prompt Engineering을 넘어서..

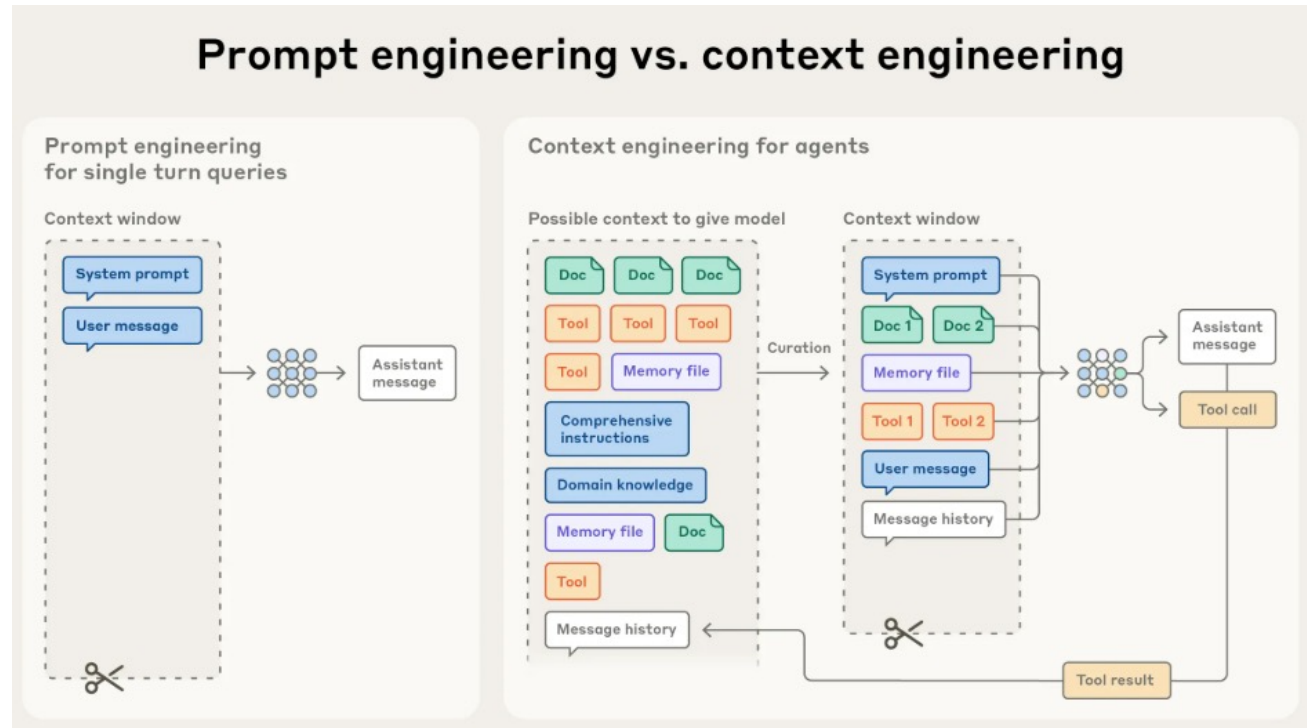
Concept

- LLM의 성능은 어떤 정보를 어떤 순서로 어떤 형태로 얼마나 적절하게 제공 하느냐에 크게 영향을 받음
- 좋은 AI 시스템 = 좋은 Context 설계

”

언어모델이 올바른 응답을 생성하도록
컨텍스트 윈도우에 들어가는 모든 정보
를 전략적으로 설계하고 제어하는 기술

”



Context Engineering의 구성 요소

무엇을 어떻게 컨텍스트에 담을 것인가?

01

System Prompt 설계

- AI의 역할·목적·제약을 정의하는 페르소나, 톤, 금지 사항 등을 명확하게 지시

02

메모리 & 히스토리

- 대화 맥락을 얼마나, 어떻게 유지할지 결정
- 중요 정보는 요약·압축하여 효율적으로 관리

03

도구 & 외부 지식

- RAG, 검색, API 결과를 선택적으로 주입
- 관련성 높은 정보만 포함하여 노이즈 최소화

04

Few-shot & 포맷

- 출력 형식과 예시 제공으로 일관된 응답 유도
- 구조화된 결과를 위한 템플릿 설계

Prompt Engineering이 "무슨 말을 할까"라면, Context Engineering은 "무엇을 알려줄까"의 설계

하네스 엔지니어링(Harness Engineering)

하네스(harness)란?

- 말(馬)에 채우는 마구(馬具).. 고삐, 재갈, 안장, 멍에..
- 강한 힘을 가진 말이 밭을 갈거나 마차를 끌 때, 그 힘을 올바른 방향으로 전달하고 폭주를 막는 장치
- LLM(말)은 강력하지만 예측 불가능함 → 하네스(마구)로 그 힘을 안전하고 유용한 방향으로 전환하자



하네스 엔지니어링(Harness Engineering)

AI 시대의 Test Harness

개념의 등장

- Terraform (DevOps 인프라 자동화 도구 ,인프라를 코드로 관리하자)의 창업자, Mitchell Hashimoto
- AI 에이전트 인프라 자동화에도 같은 개념을 가져올 수 있을까?
- 26.02 Engineer the Harness 개념 소개

”

나는 이걸 '하네스 엔지니어링'이라 부르기 시작했다.

핵심은, 에이전트가 실수를 할 때마다 그 실수를 다시는 할 수 없도록 시스템을 엔지니어링하는 것이다.

I've grown to calling this 'harness engineering.'

It is the idea that anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again

”

하네스 엔지니어링(Harness Engineering)

컨텍스트 엔지니어링이 해결하지 못하는 것

- ✓ 도구가 어떻게 호출되고 실행되는지
- ✓ 실패가 어떻게 처리되는지
- ✓ 같은 실수가 다음 세션에 또 나오지 않게 하는 법
- ✓ 누가 어떤 권한을 갖는지
- ✓ 결과물이 합격선을 넘었는지를 누가 판정하는지

하네스 엔지니어링(Harness Engineering)

핵심 개념

에이전트가 자기 결과를 인간처럼 인지할 수 있게 만들자 (Application Legibility)

- Legibility(가독성)은 "사람이 읽을 수 있다"가 아니라 "에이전트가 자기 작업물을 직접 검사할 수 있다"를 의미
- Ex) 에이전트에게 "코드 잘 짰어?"라고 묻는 게 아니라, "지금 만든 서비스의 시작 시간을 직접 측정해서 500ms 미만이면 통과"라고 숫자로 검사를 시키는 것

강제된 계층 아키텍처를 가지게 하자 (Mechanical Hierarchy)

- 코드를 작성하는 Task라면 의존성 방향을 물리적으로 강제
- Types → Config → Repo → Service → Runtime → UI
- 흐름을 어기면 Pull Request/Commit 거부 → 사람이 끼어들기 전에 시스템이 먼저 차단

한번에 다 보여주지 마라 (Progressive Disclosure)

- 거대한 5,000줄 짜리 AGENTS.md : 모든 규칙을 한 파일에 넣기 → 컨텍스트 윈도우를 다 차지하고, 모델은 "정작 중요한 부분"을 놓침
- AGENTS.md는 100줄 이내의 "지도"로만 사용하라 / 실제 상세 정보는 디렉토리에 분산
- 에이전트는 그때그때 필요한 부분만 찾아 읽음

하네스 엔지니어링(Harness Engineering)

작업 구조 예시

```
ai-harness/  
├── AGENTS.md  
├── docs/  
│   ├── mcp_usage_rules.md  
│   ├── source_rules.md  
│   ├── style_guide.md  
│   └── brief_review_checklist.md  
├── templates/  
│   ├── design_spec.md  
│   └── template.md  
├── tools/  
│   └── build_ai.py  
├── outputs/  
│   ├── ai_brief_v1.md  
│   └── ai_brief_v2.md  
├── logs/  
│   ├── search_log.md  
│   ├── source_log.md  
│   └── revision_log.md
```



수고하셨습니다.

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.